**Faculty of Software Engineering and Computer Systems**

# Programming

## Lecture #0
## Introduction and more.

Instructor of faculty
Pismak Alexey Evgenievich
Gavrilov Anton Valerievich
Kronverksky Pr. 49, 1331 room
pismak@itmo.ru
avgavrilov@itmo.ru

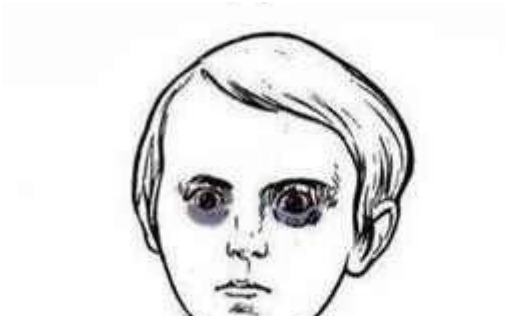
Saint-Petersburg

# Class Introduction

- Who are your teacher(s)
  - Experience with Java
  - Experience with ITMO
  - Experience with IT
- Communication (tg chat)

# Introduction. Course objectives

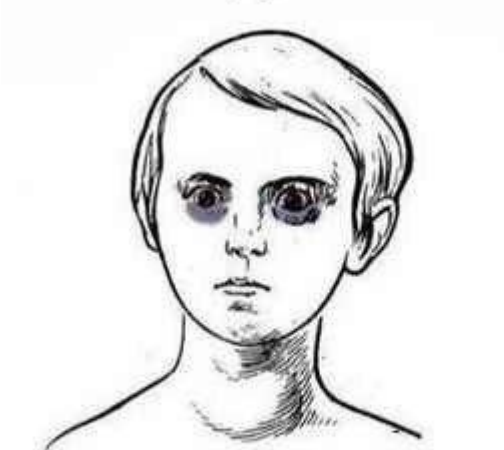
- Learn Java from basics to advanced technology
- In this course, you will acquire programming skills at a sufficient level to work in IT
- After this course, you will be able to stay awake for two nights in a row and feel ~~great~~



# Introduction. Course objectives

After this course, you should be able to do the following:

- Create simple and complex Java-applications
- Learn to apply programming paradigms such as inheritance, encapsulation, and polymorphism. This skill is useful in programming in any language
- Learn to compile, build and run Java applications of any complexity
- …
- stay awake for two nights in a row and feel ~~great~~



# Introduction. Prerequisites

To successfully complete this course, you must know:

- basics of programming and informatics
- about two hours of time each day for this course
- how to stay awake **one** night and feel great

# Introduction. Grades

# Introduction. Materials and news

se.ifmo.ru/courses/programming

tg channel

# Introduction. Lectures & guides (partly)

* Если сможешь

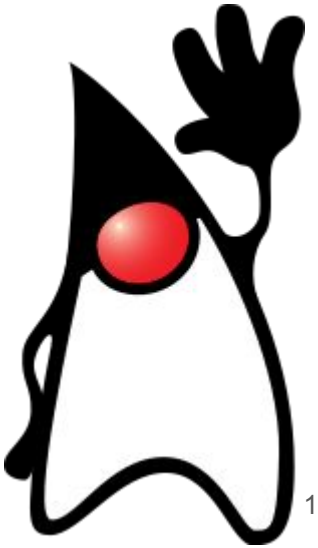
# Introduction. Course parts

- How many labs
- How many tests
- Lectures
- Exams

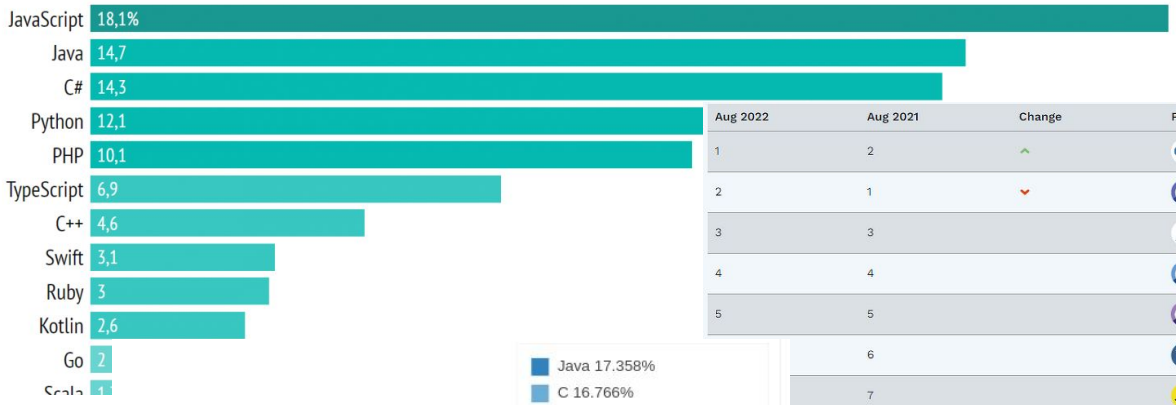
# Introduction. Tools

- Text editor (SublimeText, Notepad++)
- JDK
- IDE (intelliJ IDEA, Eclipse, NetBeans)
- CMD/Terminal (Putty, WinSCP, shell, helios)

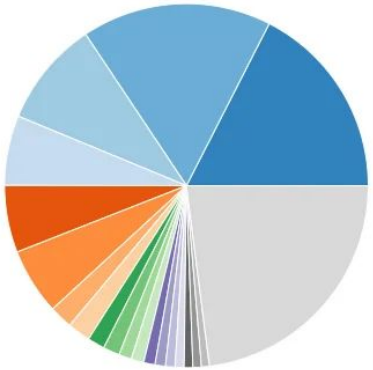
Why Java?



# Introduction. Why Java?




| Aug 2022 | Aug 2021 | Change | Programming Language | Ratings | Change |
|---|---|---|---|---|---|
| 1 | 2 | ^ | Python | 15.42% | +3.56% |
| 2 | 1 | v | C | 14.59% | +2.03% |
| 3 | 3 | | Java | 12.40% | +1.96% |
| 4 | 4 | | C++ | 10.17% | +2.81% |
| 5 | 5 | | C# | 5.59% | +0.45% |
| | 6 | | Visual Basic | 4.99% | +0.33% |
| | 7 | | JavaScript | 2.33% | -0.61% |
| | 9 | ^ | Assembly language | 2.17% | +0.14% |
| | 10 | ^ | SQL | 1.70% | +0.23% |
| | 8 | v | PHP | 1.39% | -0.80% |



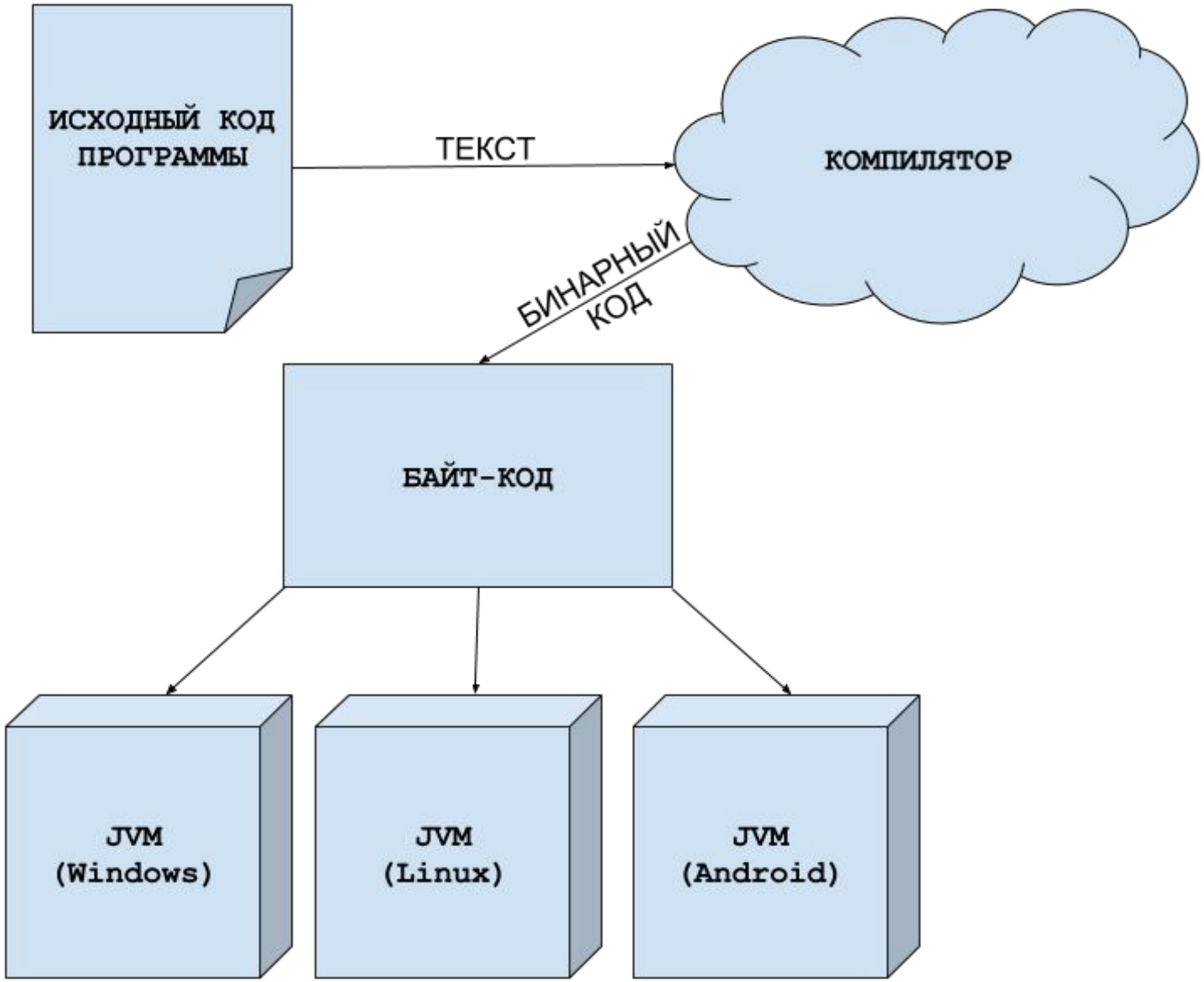
- Java 17.358%
- C 16.766%
- Python 9.345%
- C++ 6.164%
- C# 5.927%
- Visual Basic .NET 5.862%
- JavaScript 2.060%
- PHP 2.018%
- SQL 1.526%
- Swift 1.460%
- Go 1.131%
- Assembly language 1.111%
- R 1.005%
- D 0.917%
- Ruby 0.844%
- MATLAB 0.794%
- PL/SQL 0.764%
- Delphi/Object Pascal 0.748%
- Perl 0.697%
- Другие языки 22.815%

| Rank | Language | Type | Score |
|---|---|---|---|
| 1 | Python | | 100.0 |
| 2 | Java | | 95.3 |
| 3 | C | | 94.6 |
| 4 | C++ | | 87.0 |
| 5 | JavaScript | | 79.5 |
| 6 | R | | 78.6 |
| 7 | Arduino | | 73.2 |
| 8 | Go | | 73.1 |
| 9 | Swift | | 70.5 |
| 10 | Matlab | | 68.4 |

# Introduction. Why Java?

- high threshold for entry into IT (low competition)
- Java is a very large and complex product. Knowing Java is easy to master other areas
- cross-platform

# Introduction. Cross-platform

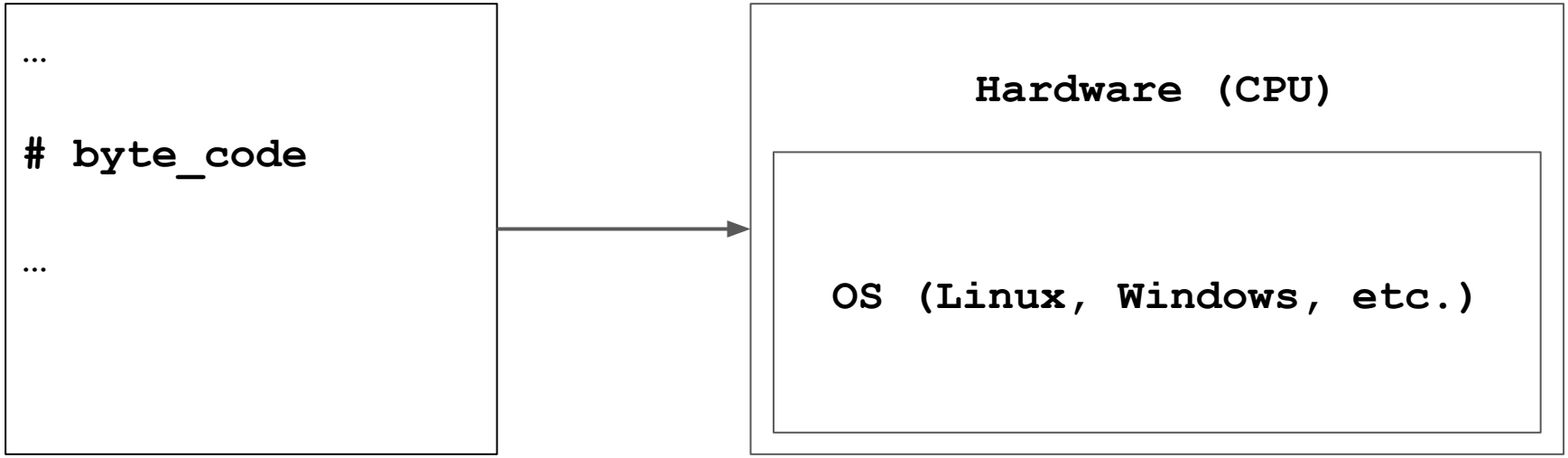


# Introduction. Cross-platform

```
public class Object
   public Object() {}

   …
}
```

```
Compiled from "Object.java"
public class java.lang.Object
{
  public java.lang.Object();
    Code:
       0: return
  public boolean
equals(java.lang.Object);
    Code:
       0: aload_0
       1: aload_1
       2: if_acmpne     9
       5: iconst_1
       6: goto          10
       9: iconst_0
      10: ireturn
  …
```

# Introduction. Cross-platform



# Java

# Java. Comments

Однострочные комментарии

```
// комментарии
```

Многострочные комментарии

```
/*
комментарии
*/
```

Специальные комментарии (javadoc)

```
/**
*
* @author  James Gosling
*/
```

# Java. First application

***Main.java***

```
1.  public class Main {
2.
3.      public static void main(String[] args) {
4.
5.          System.out.println("Внимание!");
6.
7.      }
8.
9.  }
```

# Java. Simple math-application

```
1.  public class SimpleProgram {
2.
3.      public static void main(String[] args) {
4.
5.          var x = 5.0D;
6.
7.          var result = Math.pow(x, 2.5);
8.
9.          System.out.println(result);
10.
11.     }
12.
13. }
```

# Java. Program is written. What's next?

JDK
- compiler
- debugger
- byte-code tools
- JVM
- standard library

# Java. Program is written. What's next?

1. Compilation

```
javac   Main.java
javac   -d   target   Main.java
javac   --help
```

2. Packaging to jar

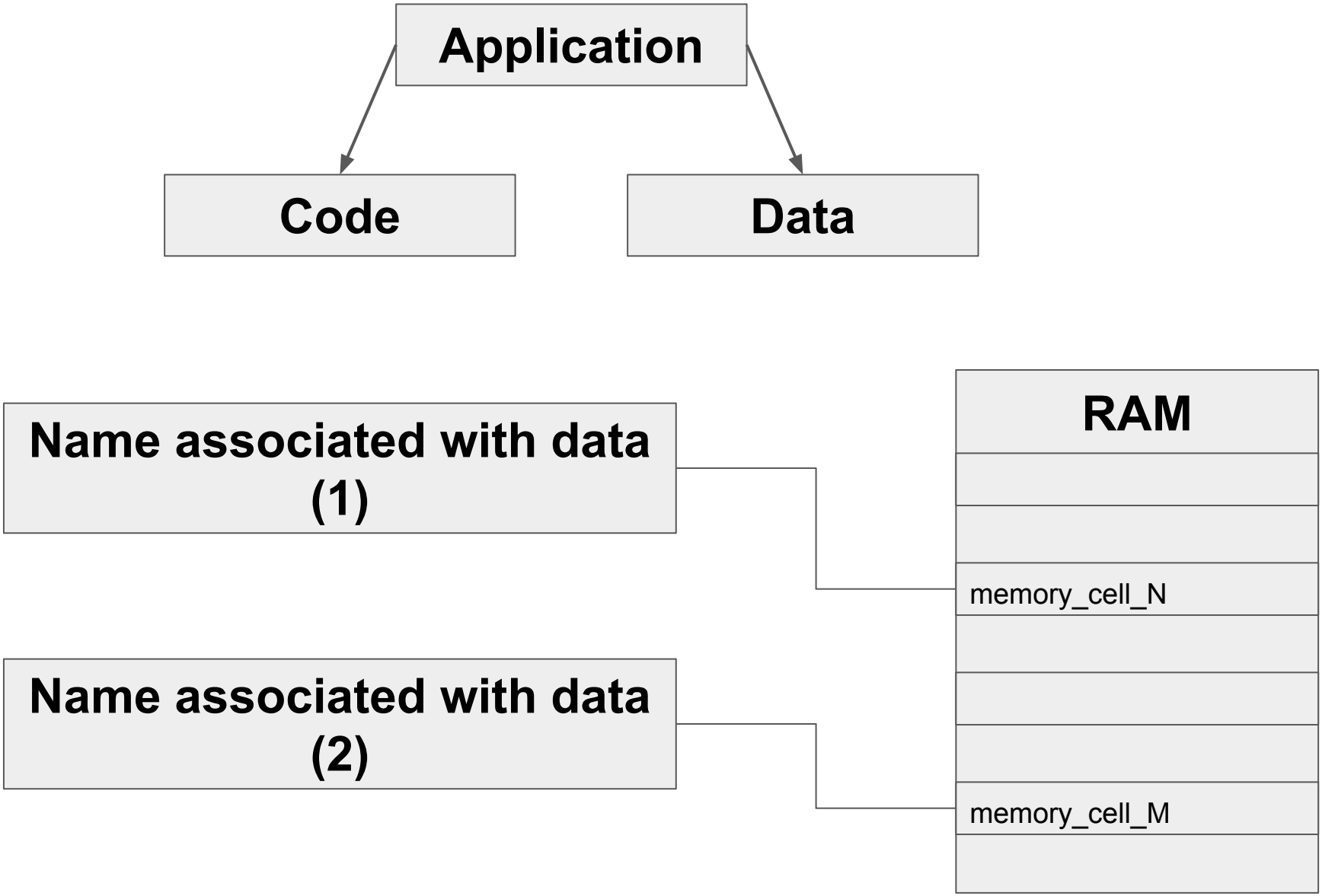
```
jar   -cfm   app.jar   MANIFEST.mf   *.class
```

3. Run

```
java   -jar   app.jar
java   Main
```

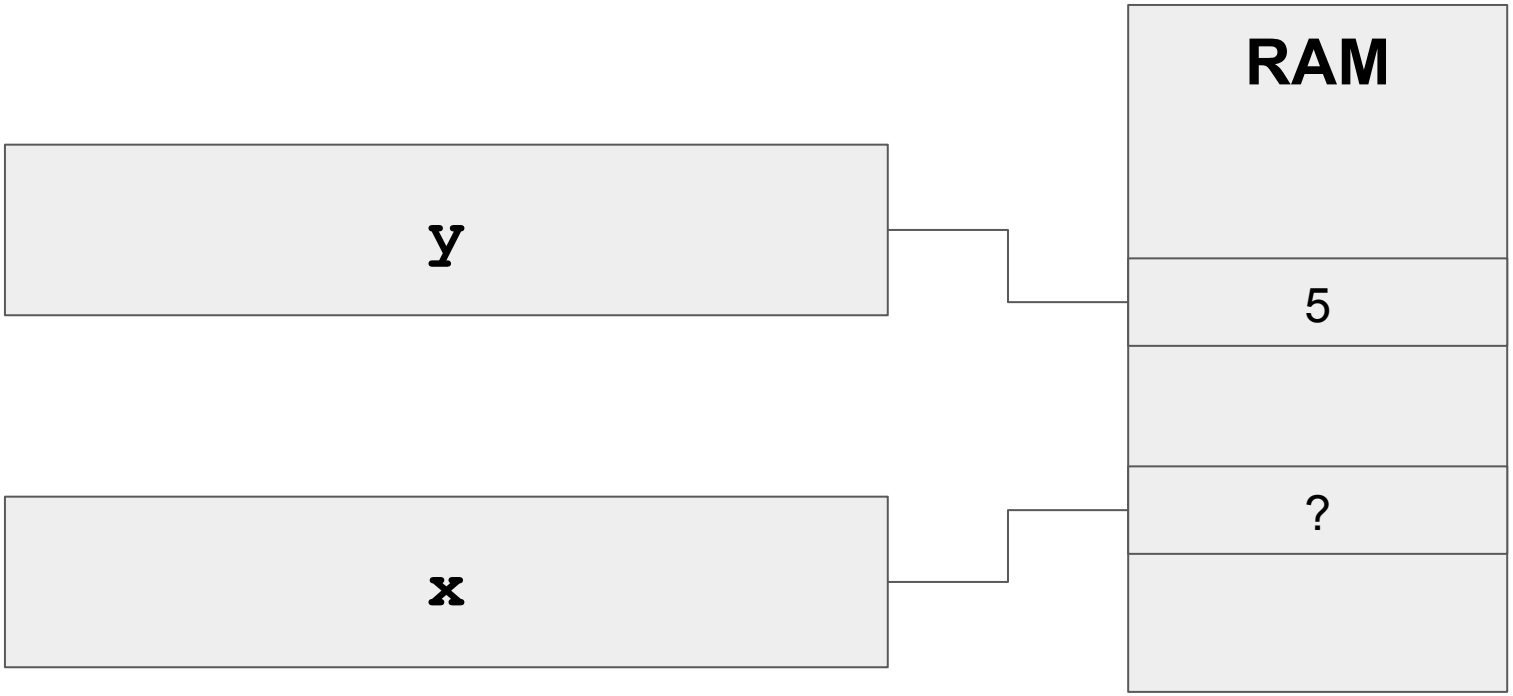
4. Debugging

# Java. Variables and constants



# Java. Variables and constants

```
int x;

int y = 5;


data_type variable_name [ =  default_value ] ;
```

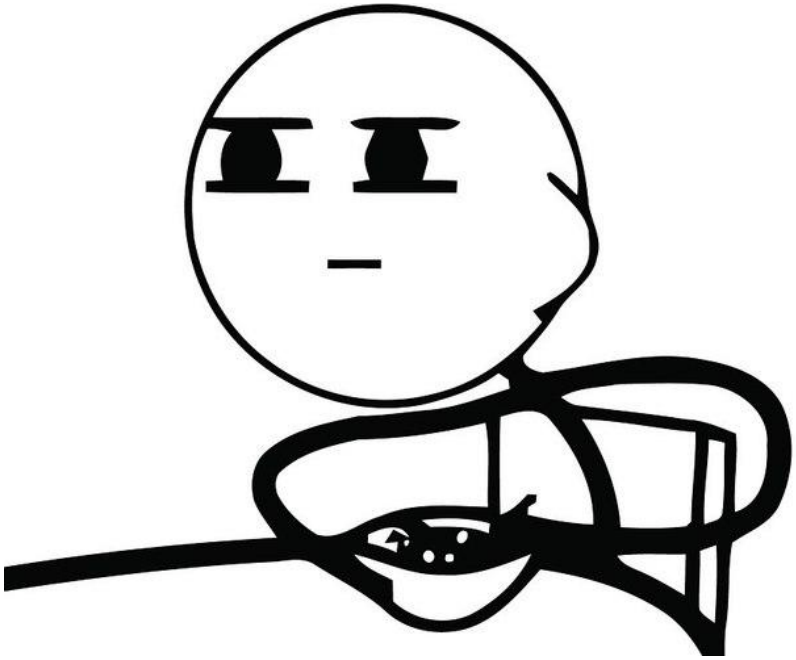


# Java. Variables and constants

```
final double E = 5.0;

final double R;
// …
R = 5.0;
```

<u>what is it for?</u>

```
int total = 8 * 5 * 4 * 12;
```

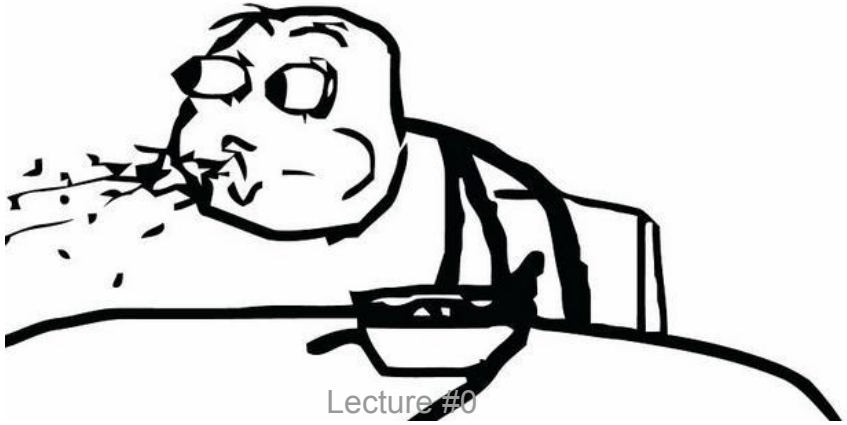


# Java. Variables and constants

**what is it for?**

```
int total = 8 * 5 * 4 * 12;  // BAD
```

```
final int HOURS = 8, DAYS = 5, WEEKS = 4, MONTHS = 12;
// …
int total = HOURS * DAYS * WEEKS * MONTHS;   // GOOD
```



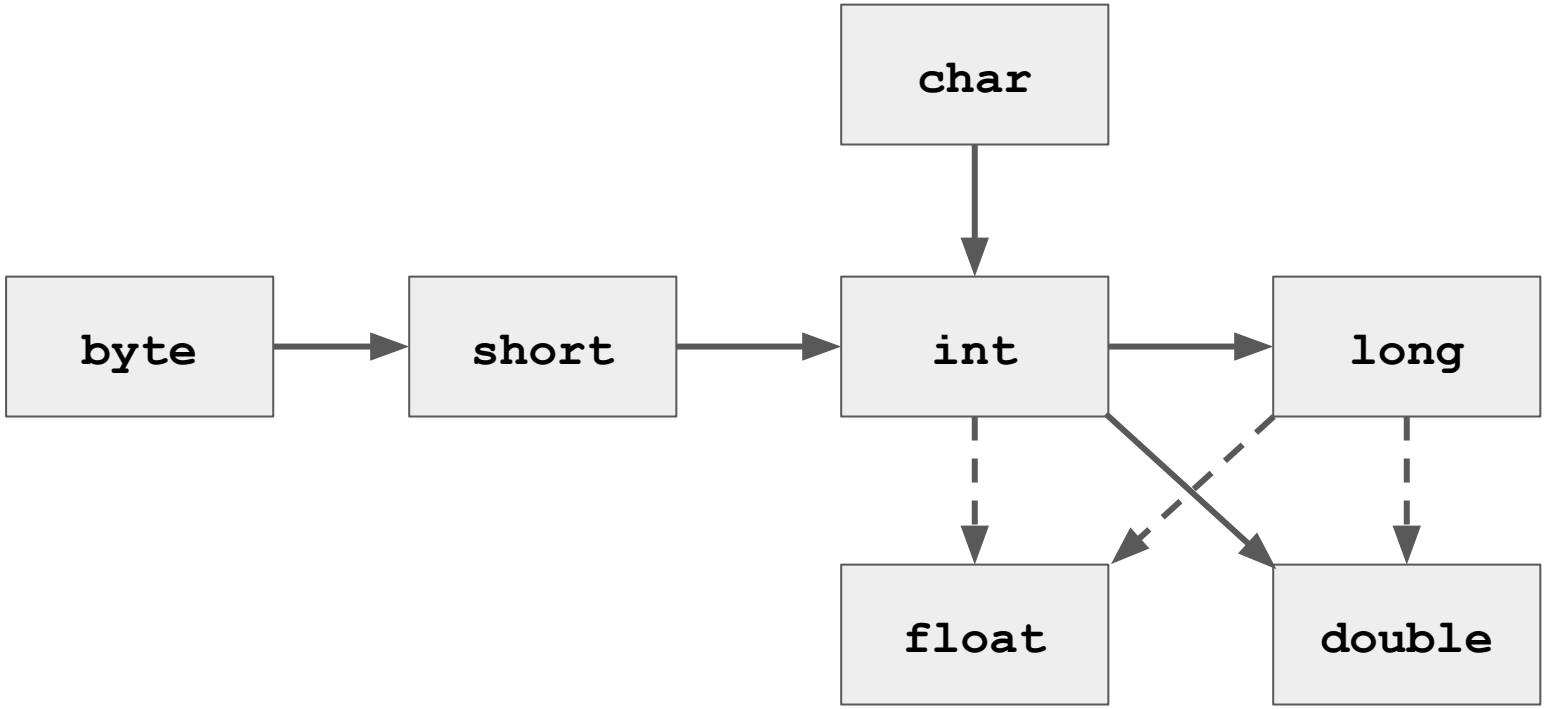
# Java. Data types

| Целочисленные | Вещественные | Логические | Объекты |
|---|---|---|---|
| byte | double | boolean | |
| short | float | | |
| char | | | |
| int | | | |
| long | | | |

# Java. Data types

| Data type | Size (bit) | Values |
|---|---|---|
| byte | 8 | от -128 до 127 |
| short | 16 | от -32768 до 32767 |
| char | 16 | от 0 до 65535 |
| int | 32 | от -2147483648 до 2147483647 |
| long | 64 | от -9223372036854775808<br>до 9223372036854775807 |
| float | 32 | от -1.4e-45f до 3.4e+38f |
| double | 64 | от -4.9e-324 до 1.7e+308 |
| boolean | 1 or 32 | true or false |

# Java. Convert data types



# Java. Casting data types

```
double e = 2.7;

int notE = (int) e;    // notE = 2


int roundE = (int) Math.round(e);    // roundE = 3
```

# Java. Operators

| Unary | Binary |
| --- | --- |
| - | - + |
| -- | / * |
| ++ | \| \|\| |
| ~ | & && |
| ! | == != |
| [] | |

```
int x = -y;          // унарный "минус"

int r = x - y;       // бинарный "минус"
```

# Java. Operators

| № | Operator |
|---|---|
| 1 | [] . () |
| 2 | ! ~ ++ -- + - (cast) new |
| 3 | * / % |
| 4 | + - (binary) |
| 5 | >> << >>> |
| 6 | < <= > >= instanceof |
| 7 | == != |
| 8 | & |
| 9 | ^ |
| 10 | \| |
| 11 | && |
| 12 | \|\| |
| 13 | ?: |
| 14 | = += -= *= /= %= \|= ^= <<= >>= >>>= |

# Java. Math functions

| Функция | Конструкция |
|---|---|
| Абсолютное значение x | Math.abs ( x ); |
| Косинус x | Math.cos ( x ); |
| Синус x | Math.sin ( x ); |
| Экспонента | Math.exp ( x ); |
| Квадратный корень x | Math.sqrt ( x ); |
| Корень суммы квадратов (гипотенуза) | Math.hypot ( x, y ); |
| Натуральный логарифм | Math.log ( x ); |
| Округление до ближайшего | Math.round ( x ); |
| Возведение ‘x’ в степень ‘y’ | Math.pow( x, y ); |

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Math.html

# Java. Math constants

- `Math.PI` // 3.14159265358979323846
- `Math.E` // 2.71828182845904523 54

```
double res1 = 3 * Math.PI / 2;

double res2 = Math.log(5) * Math.E;
```

# Java. Just in time test

```
1.  public class PracticValiables {
2.
3.      public static void main(String[] args) {
4.
5.          byte x = 127;
6.
7.          x++;
8.
9.          System.out.println(x);
10.
11.     }
12. }
```

**128** **1** **-1** **127**

# Java. Strings

```
String name = "Alis";
```

```
String lastName = new String("Parker");
```

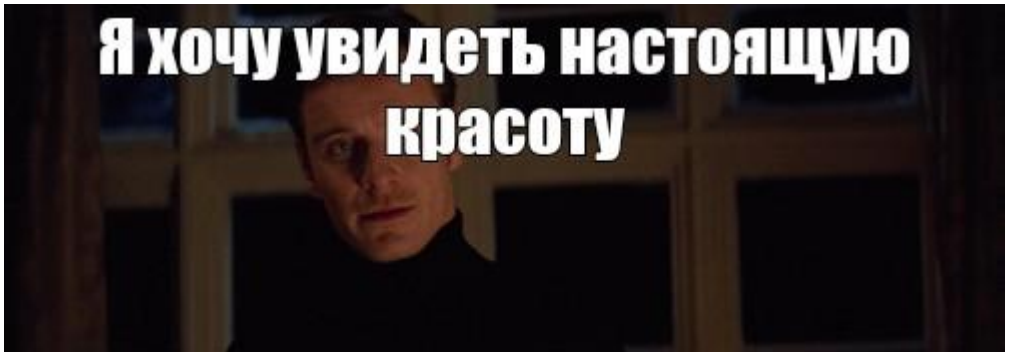
```
String a = "Java\u2122";          // Java™
String b = "";                    // пустая строка
```

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html

# Java. Formatting output

```
double x = 5.683;

double y = Math.pow(x, 2.5);

System.out.printf("%.2f ^ 2.5 = %.3f", x, y);
```

```
Output:   5,68 ^ 2.5 = 76,890
```



# Java. Formatting output

```
printf(String format, Object... args)
```

```
printf("%.4f", Object... args)
```

What is "format"?

```
%[argument_index$][flags][width][.precision]conversion
```

# Эльфийско-русский словарь

`%[argument_index$][flags][width][.precision]conversion`

**argument_index$** — целое десятичное число, указывающее позицию аргумента в списке аргументов

**flags** — специальные символы для форматирования. Например, флаг "+" означает, что числовое значение должно включать знак +, флаг "-" означает выравнивание результата по левому краю, флаг «,» устанавливает разделитель тысяч у целых чисел.

**[width]** — положительное целое десятичное число, которое определяет минимальное количество символов, которые будут выведены

**[.precision]** — неотрицательное целое десятичное число с точкой перед ним. Обычно используется для ограничения количества символов.

**conversion** — это символ, указывающий, как аргумент должен быть отформатирован. Например **d** для целых чисел, **s** для строк, **f** для чисел с плавающей точкой.



*** copy-paste from Java documentation**

# Java. Summary about you next steps

- You will do the lab work at home
  - Write code
  - Copy it to server (helios.cs.ifmo.ru)
  - Compile and run it
- Your teacher may give you additional practice tasks
- Examination of lab work
  - Be ready to hear the question, made up from unfamiliar words
  - Try to find answer and repeat discuss with teacher

**Faculty of Software Engineering and Computer Systems**

# Programming

## Lecture #1
## Syntax constructions. Methods. Arrays.

Instructor of faculty
Pismak Alexey Evgenievich
Gavrilov Anton Valerievich
Kronverksky Pr. 49, 1331 room
pismak@itmo.ru
avgavrilov@itmo.ru

Saint-Petersburg

# Syntax constructions

# Statements vs Expressions

```
1.  System.out.println("!") ;     // method invocation statement
2.  if ( a > 0 ) z = y + a ;      // if statement
3.  while (true) counter++ ;      // while statement
4.  f = a*x*x + b*x + c ;         // assignment statement
```

```
1.  (float) Math.cos(x)           // cast expression
2.  a * ( x + b) > 0              // relational expression
3.  isDone && ! hasErrors         // logical expression
4.  a * x * x + b * x + c         // arithmetic expression
```

# Conditional statement

```
if ( condition ) statement


if ( condition ) statement else statement_2


if ( condition ) statement else if ( condition ) statement_x
...
```

```
final int TEMPERATURE_LIMIT = 25;
int currentTemperature = 21;
boolean isSwitchedOff = false;

if(currentTemperature > TEMPERATURE_LIMIT) {
    …
    isSwitchedOff = true;
}
```

# Dangerous!

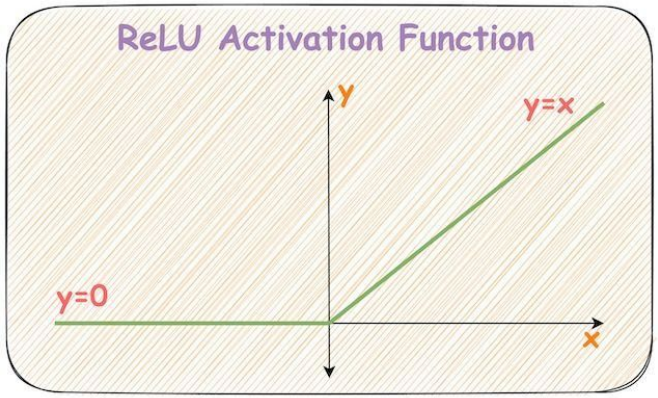
```
boolean conditional = a * b > c;

if ( conditional ) { … }
```

```
if ( conditional == true ) { … }
if ( conditional != false ) { … }
```

```
if ( String.valueOf(conditional).equals("true") )
if ( String.valueOf(conditional).length() == 4 )
if ( conditional == true && conditional != false )
```

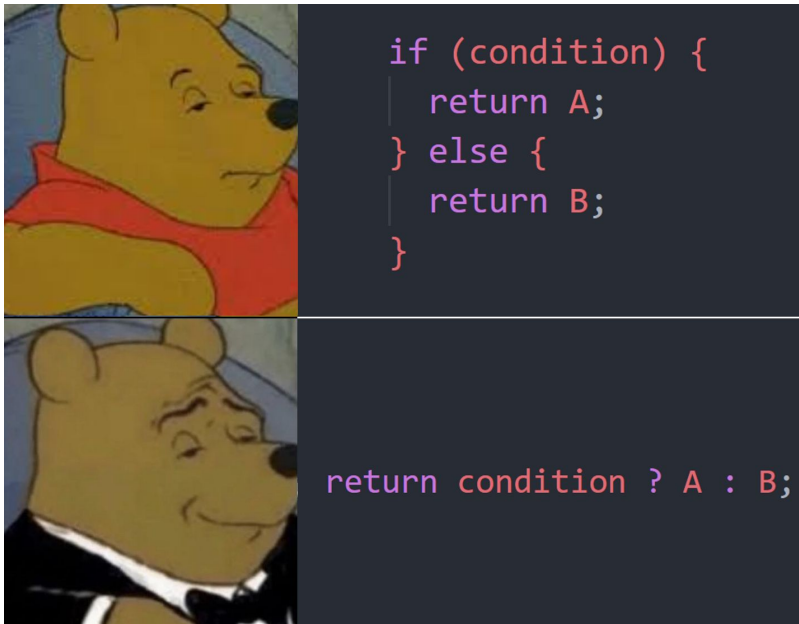
# Ternary operator

```
condition ? expression : expression;

// expression must return value
```

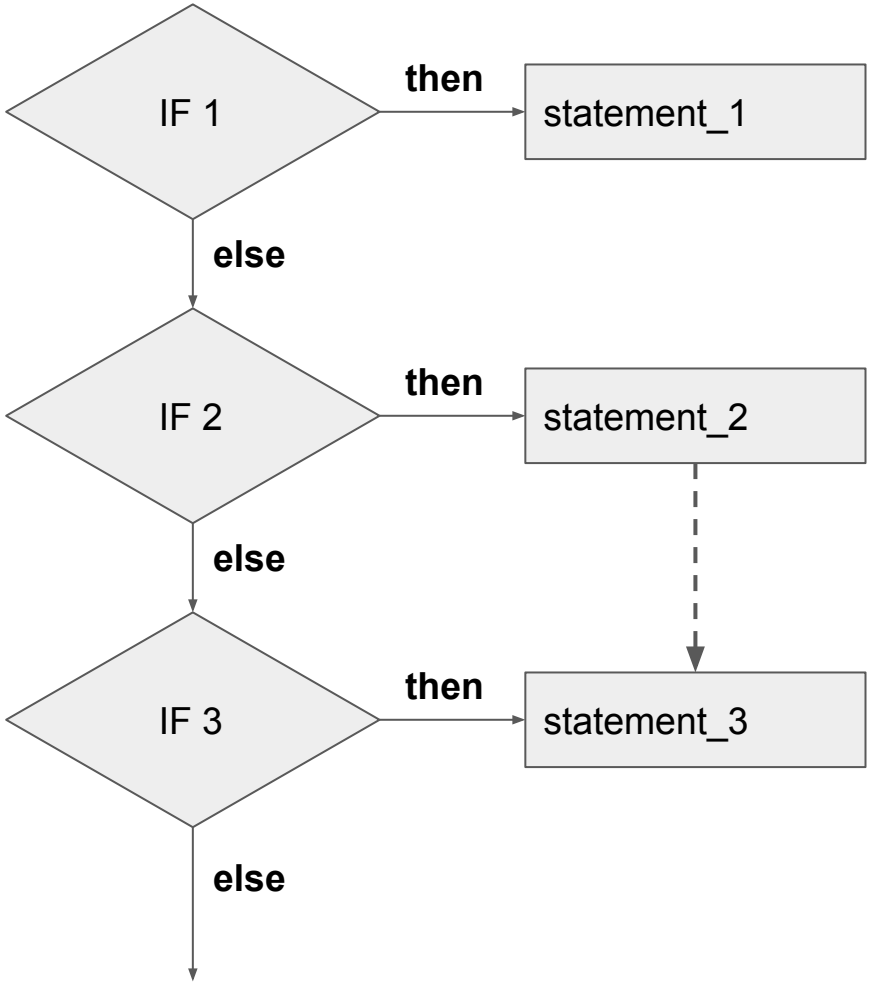



```
int relu = x > 0 ? x : 0;
```

```
int relu;
if(x > 0) {
  relu = x;
} else {
  relu = 0;
}
```

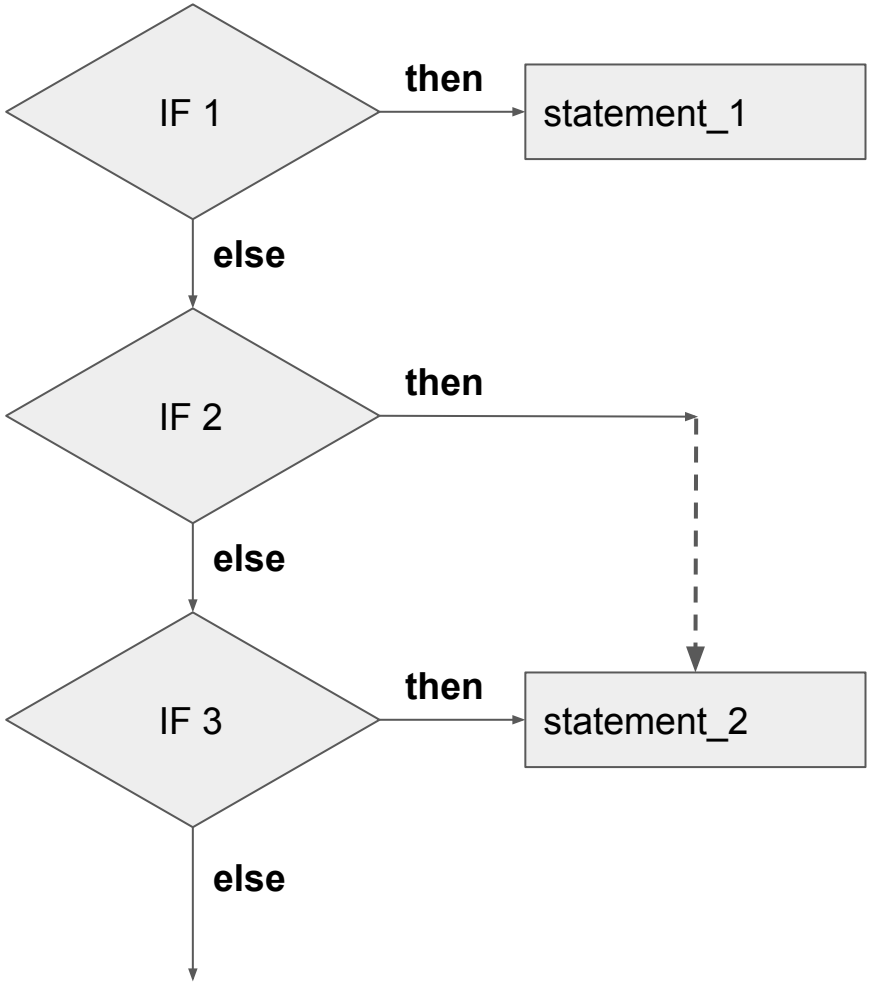


# Multivariate branching (classic)



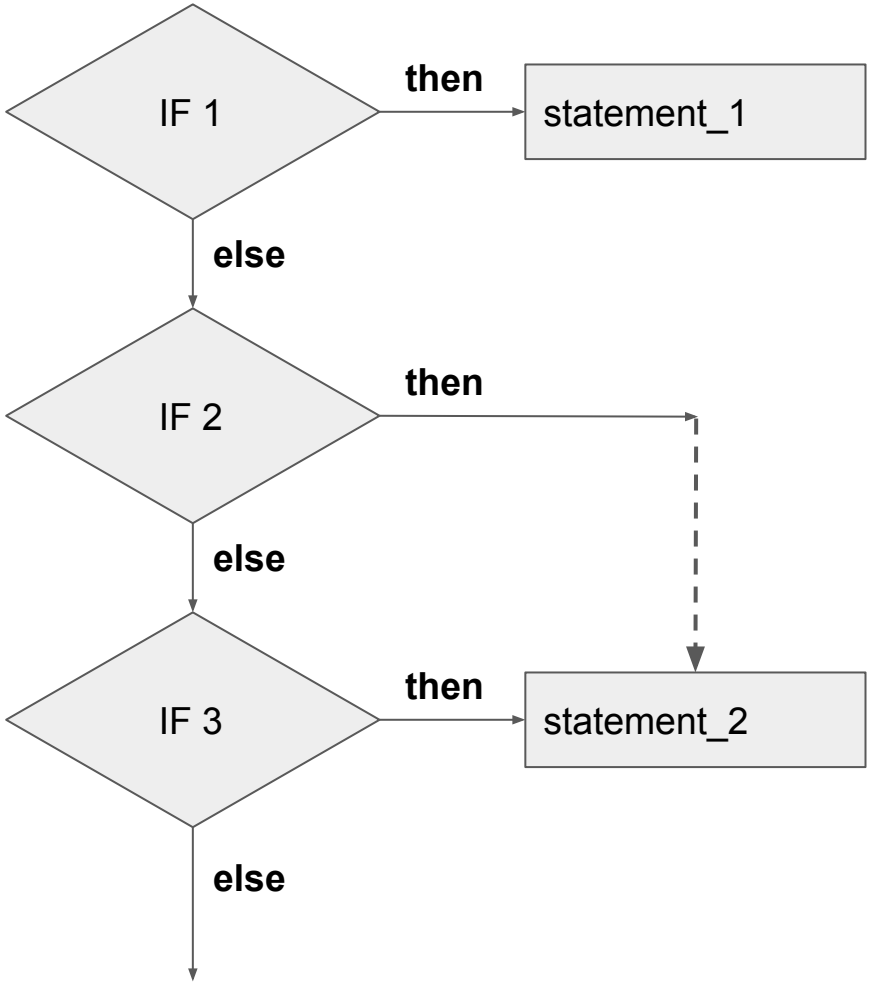

```
switch ( x ) {

case 1 : statement_1;
    break;

case 2 :
case 3 : statement_2;

default : statement_n;

}
```

# Multivariate branching (upgrade#1)



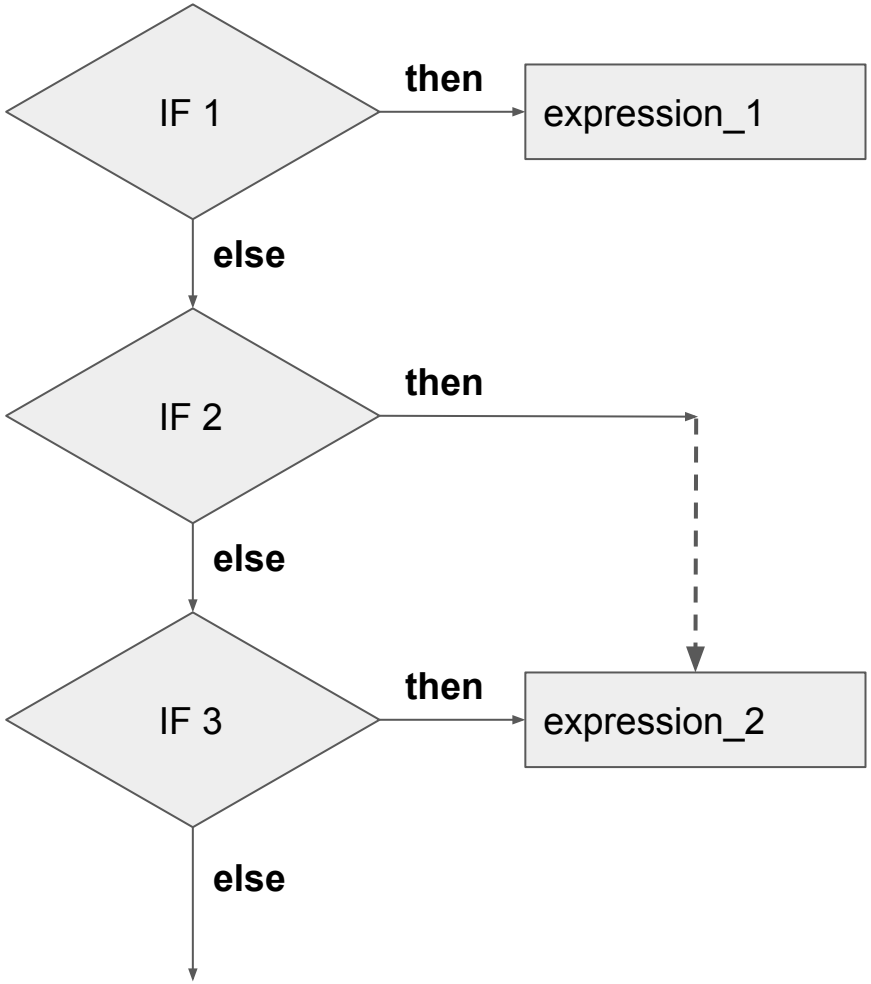

```
switch ( x ) {

case 1 : statement_1;
    break;

case 2,3 : statement_2;

default : statement_n;

}
```

# Multivariate branching (upgrade#2)



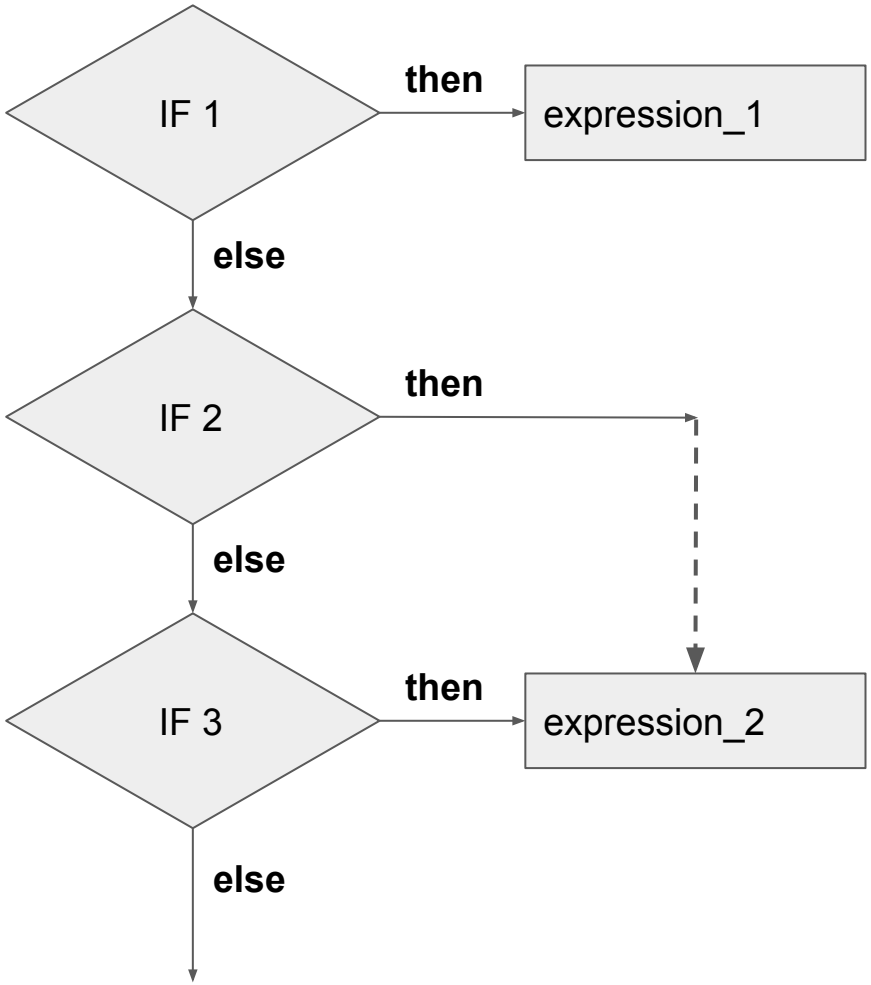

```
switch ( x ) {

case 1 -> statement_1;

case 2,3 -> statement_2;

default -> statement_n;

}
```

# Multivariate branching (upgrade#3)




```
var y = switch ( x ) {

case 1 -> 10;

case 2,3 -> 20;

default -> 100;

};
```

# Multivariate branching (upgrade#4)




```
var y = switch ( x ) {

    case 1 -> 1;

    case 2,3 -> {
        // calc result
        yield result;
    };

    default -> 10;

};
```

# Indefinite loops

```
while ( condition ) statement;



while ( true ) {

  // do something

}



while ( x > 0 ) {      // this may never executed

    // code here

}
```

# Indefinite loops

```
do statement while ( condition );
```

```
do {
  // do something
} while ( true );
```

```
do {      // executed at least once
   // code here
} while ( x > 0 );
```

# Definite loop `for`

```
for ( init_block; condition; calc_block ) statement;


/*
 * Print numerals
 */
for (int i = 0; i < 10; ++i) {

    System.out.println( i );

}


// square table
for (int i = 0, j = 0; i < 10 && j < 10; ++i, ++j) {

    System.out.printf("%d * %d = %d", i, j, i * j );

}
```

# Definite loop `for`

```
for (;;) {

    System.out.println( “я жидкий” );

}
```

1. won’t compile
2. won’t run
3. will run once
4. other

# Iterable loop `for`

```
for ( def_variable : set ) statement;
```

```
int[] array = // initialization array ;

for (int element : array) {
    if (element != 0) {
        System.out.print(element);
    }
}
```

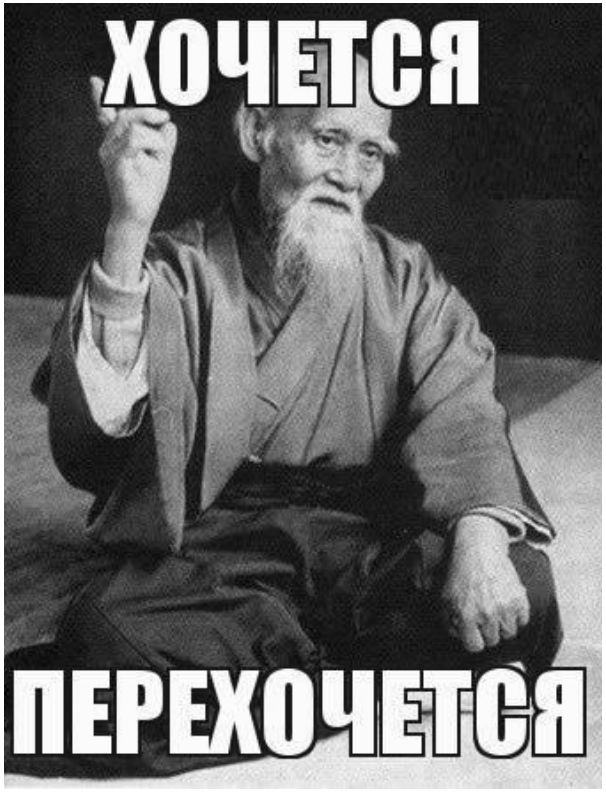
# Interrupt execution

```
1.  String str = "some string";
2.
3.  for (char currentChar : str.toCharArray()) {
4.
5.      for(int i = 0; i < 2; ++i) {
6.
7.          if ((int)currentChar == i) {
8.
9.              break;
10.
11.         }
12.     }
13. }
```

# Interrupt execution

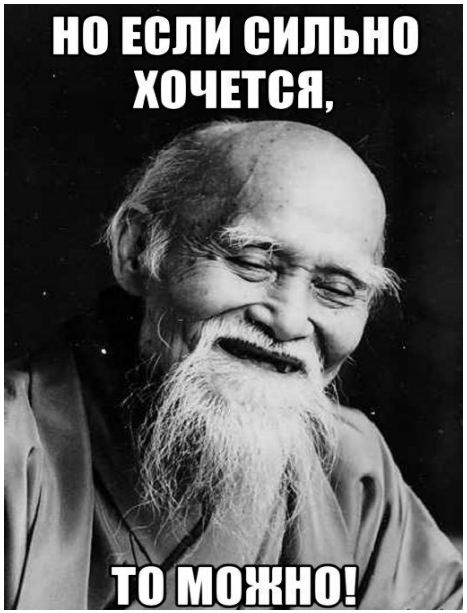
```
1.  String str = "some string";
2.
3.  for (char currentChar : str.toCharArray()) {
4.
5.      for(int i = 0; i < 2; ++i) {
6.
7.          if ((int)currentChar == i) {
8.
9.              continue;
10.
11.         }
12.         // ...
13.     }
14. }
```

# but… if you want to exit from all loops??




* If you want, you won’t want

# Interrupt all loops




```
String str = “some string”;

full:
for (char c : str.toCharArray()) {

    for(int i = 0; i < 2; ++i) {

        if ((int)c == i) {

            break full;

        }
        // ...
    }
}
```

# Blocks and scope

```
{
    // statements, declarations
}
```

```
if ( condition ) expression
```

→

```
if ( condition ) {
    // code
}
```

```
public static void main (String[] args) {
    // code
    {
        // code
    }
    // code
}
```

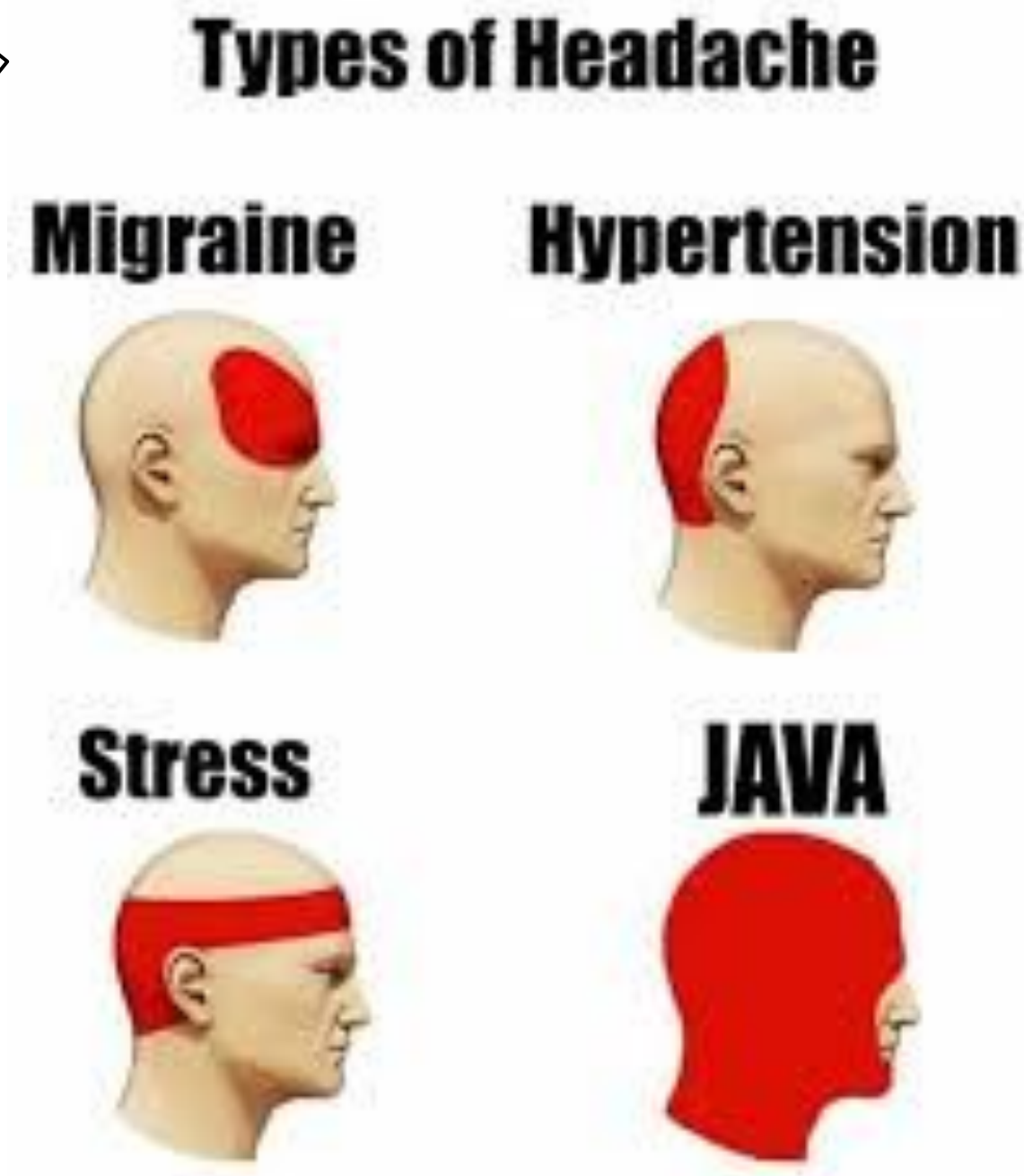
# “Subprograms” (methods)

```
public static void printMessage (String msg) {

    System.out.println (msg);

}

public static void main(String[] args) {

    printMessage (“I am liquid”);

}
```

# Methods

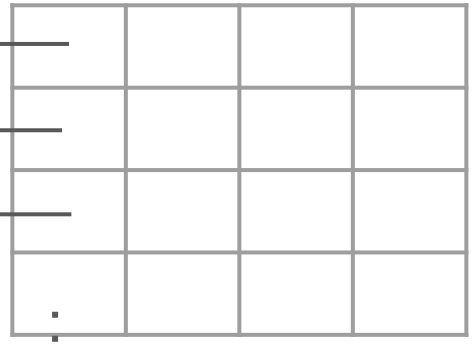
```
public static int cube (int arg) {
    return arg * arg * arg;
}

public static void main(String[] args) {
    printMessage ("5^3 = " + cube(5));
}

public static void printMessage (String msg) {
    System.out.println (msg);
}
```

Questions?



# Reference and value data types

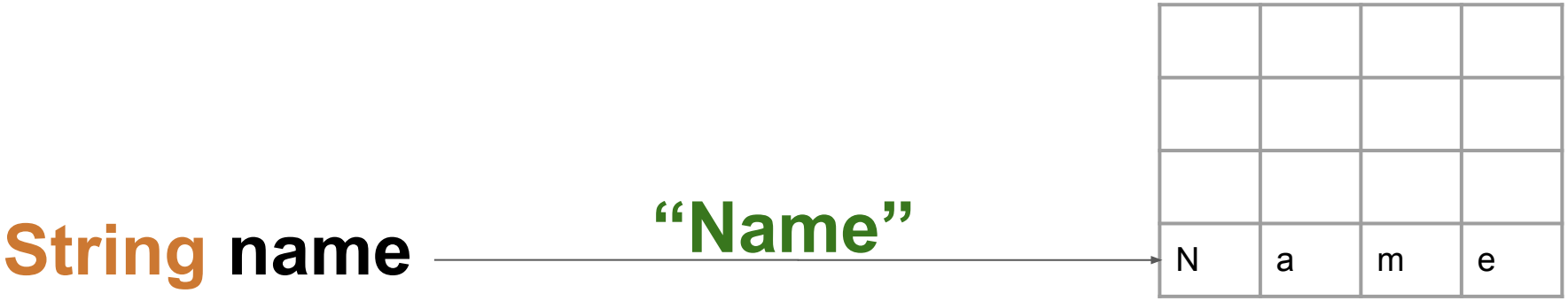
```
int x = 5;
double arg = 1.544;
char c = 'A';
```



?

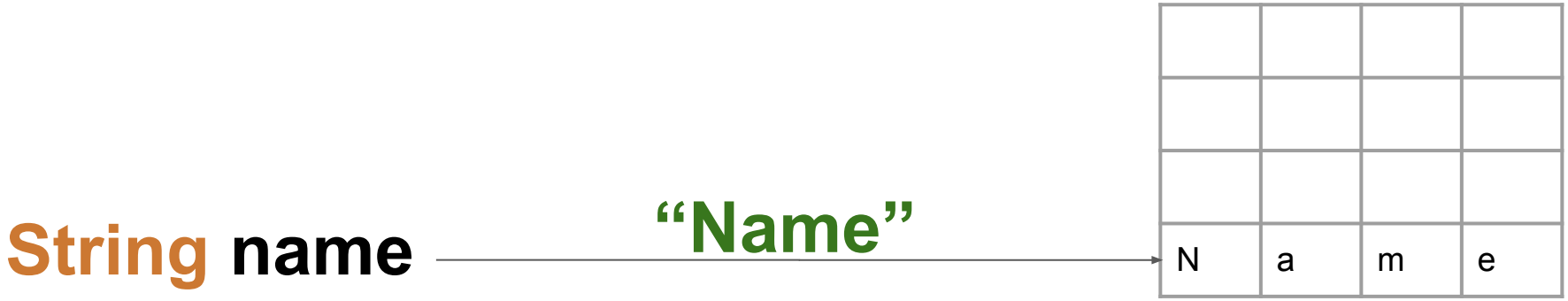
```
String name = "Name";
```

# Ref- and valuable data types




```
String name1 = new String("Name");

String name2 = new String("Name");

System.out.println ( name1 == name2 );
```

# Ref- and valuable data types




```
String name1 = new String(“Name”);
String name2 = new String(“Name”);

String name3 = “Name”;
String name4 = “Name”;
```

# Operator `new`

```
int[] y = new int[2];
```

```
String str = new String("I am liquid");
```

How reset reference variable to uninitialized value?

# Value ‘null’

```
int[] y = null;

String str = null;

y.length     // after this operations
str.trim()   // will be errors
```

# Arrays

```
int[] a;
int b[];
int[] x = {5, 2};
int[] y = new int[2];
```

# Arrays

```
int[] x = {5, 2};
int count = x.length;  // property
```

```
java.util.Arrays          // work with array
    sort
    search
    copy
```

# Arrays

**Sort:**

**Arrays.sort ( … )**

**Arrays.parallelSort ( … )**

**Search:**

**Arrays.binarySearch ( … )**

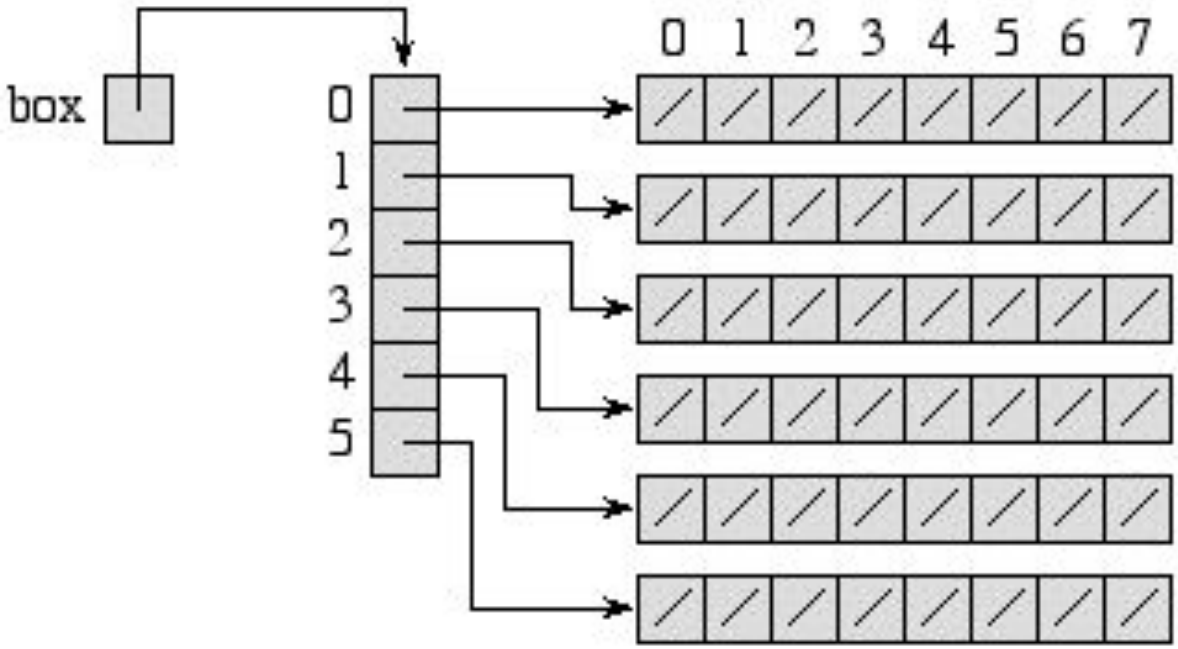
**Copy:**

**System.arrayCopy ( … )**

**Arrays.copyOfRange ( … )**

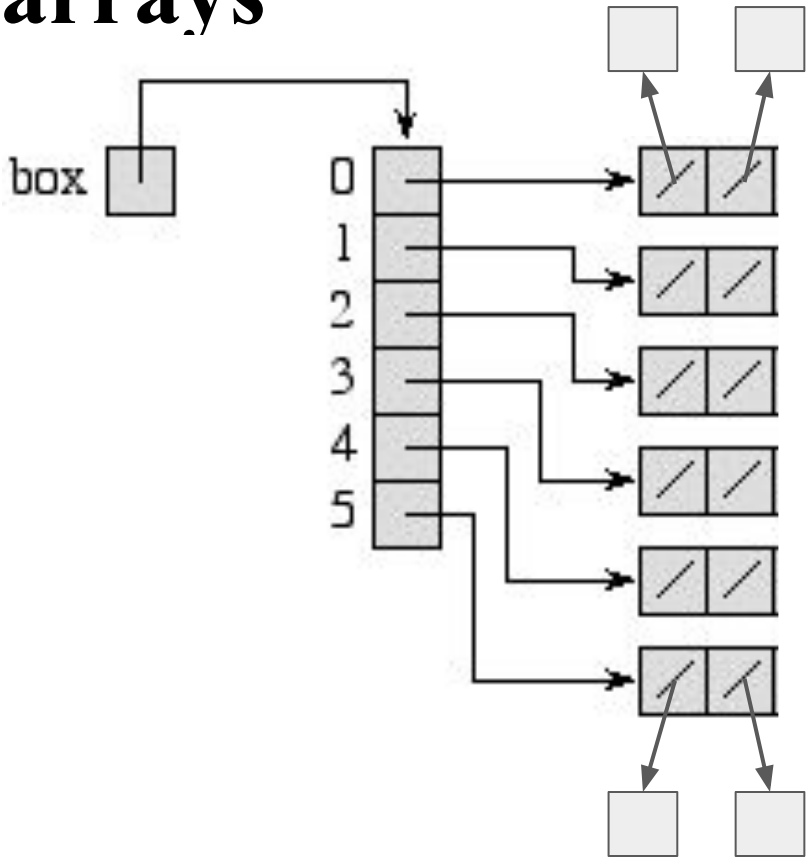
**filling, applying specific math expression, set default values etc.**

# Arrays



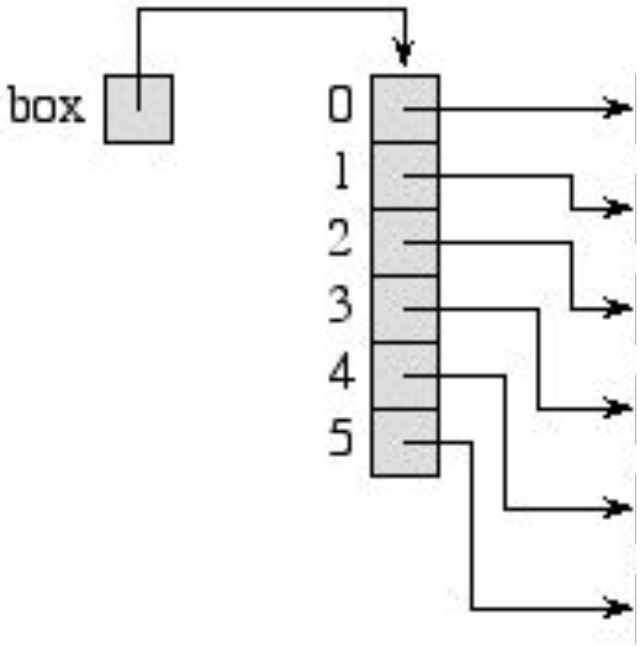

```
int[][] matrix;

int[][] box = new int[6][8];

int[][] box = { {1, 2, 3},  {4, 5, 6} };
```

# Crazy arrays




```
int[][][] box = new int[6][2][1];
```

# Arrays




```
int[][] box;

int[] box[] = new int[6][8];

int[][] box = new int[6][];

int box[][] = { {1, 2, 3},  {4, 5, 6} };
```

# How to write a method with arguments like this System.out.printf ?

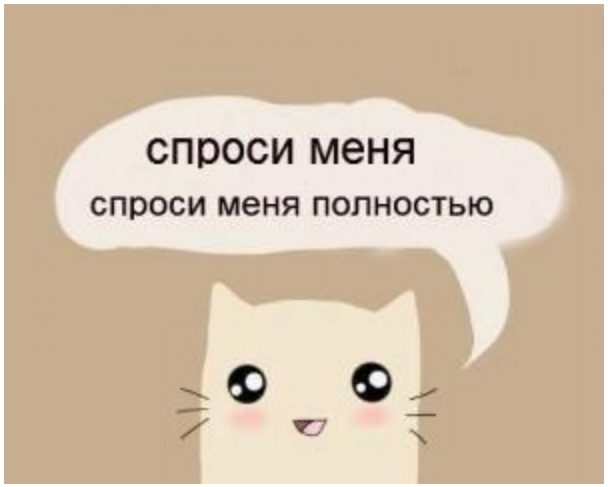
```
public static void main(String[] args) {

    System.out.printf("5^3 = %d", cube(5));

    System.out.printf("%d = %d", cube(5), 5*5*5);

}
```

String + one argument

String + two arguments

# Variable arguments (VARARGS)

```
public static void main(String[] args) {

    printMessage ("This ", "Bob");
    printMessage ("Я", "угадаю", "как", "тебя", "зовут");
}

public static void printMessage (String ... msgs) {
    for(String s : msgs) {
        System.out.println (s);
    }
}
```




* ask me questions

# Utils. Classpath & Imports

```
1.  import static java.lang.Math.*;
2.
3.  /**
4.   *  Безысходники (game of words: sources + hopelessness)
5.   */
6.  public class PracticMath {
7.
8.    public static void main(String[] args) {
9.
10.     double x = 5.1, y = 3.57;
11.
12.     double res = sin(( x + 1) / 3*PI) * 8*cos(y);
13.
14.   }
15.
16. }
```

# Utils. Classpath & Imports

```
1.  import java.util.Arrays;
2.
3.  /**
4.   *  Безысходники (game of words: sources + hopelessness)
5.   */
6.  public class PracticArrays {
7.
8.    public static void main(String[] args) {
9.
10.     int[] x = new int[2];
11.
12.     Arrays.fill(x, 10);
13.
14.   }
15.
16. }
```

**Faculty of Software Engineering and Computer Systems**

# Programming

## Lecture #2.
## Introduction to OOP.

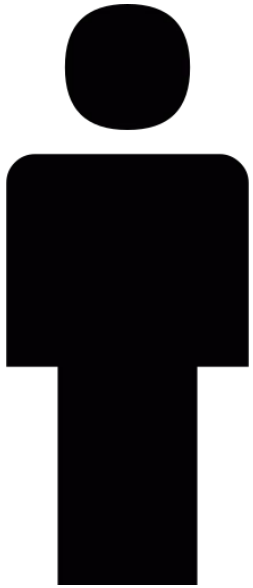
Instructor of faculty
Pismak Alexey Evgenievich
Gavrilov Anton Valerievich
Kronverksky Pr. 49, 1331 room
pismak@itmo.ru

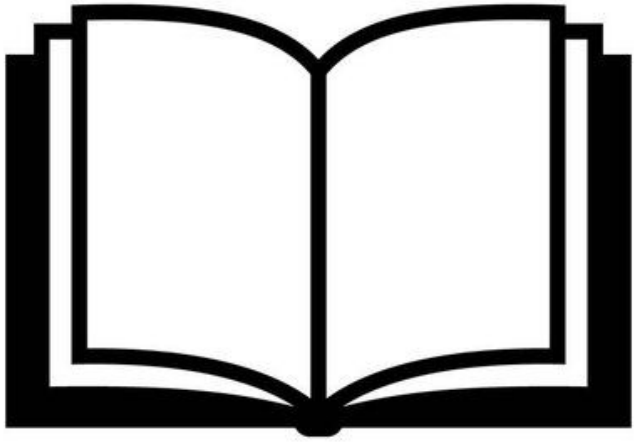
Saint-Petersburg

# OOP… What’s this and why?

1. Imperative programming
   1.1. Procedural programming
   1.2. Object-oriented programming
2. Declarative programming
   2.1. Logic programming
   2.2. Functional programming
3. Automata-based programming
4. Event-driven programming

# Objects of real world (properties & actions)





# Objects of real world…

## ...for example, humans

```
public class Main {

  public static void main(String args[]) {
    // характеристики
    String name = "Шустрый";          // имя
    int health = 625;                 // здоровье
    int mana = 820;                   // авторитет
    int age = 28;                     // возраст
    long longitude = ...;   // координаты местоположения
    long latitude = ...;    // долгота и широта

  }
}
```

# Objects of real world can…

```
public class Main {

    // main пропущен

    public static void sayPhrase() {
      System.out.println("Самые быстрые ноги на районе");
    }

    public static long runToWest(int age, int heath, long longitude) {
        speed = health * age;
        return longitude - speed;
    }
}
```

# Objects of real world can…

```
public class Main {

  public static void main(String args[]) {

     // объявление характеристик

      sayPhrase();

      if( runToWest(age, health) == 30_375399) { // в Купчино

        mana++;

      }

  }
}
```

# What if there are two of them?

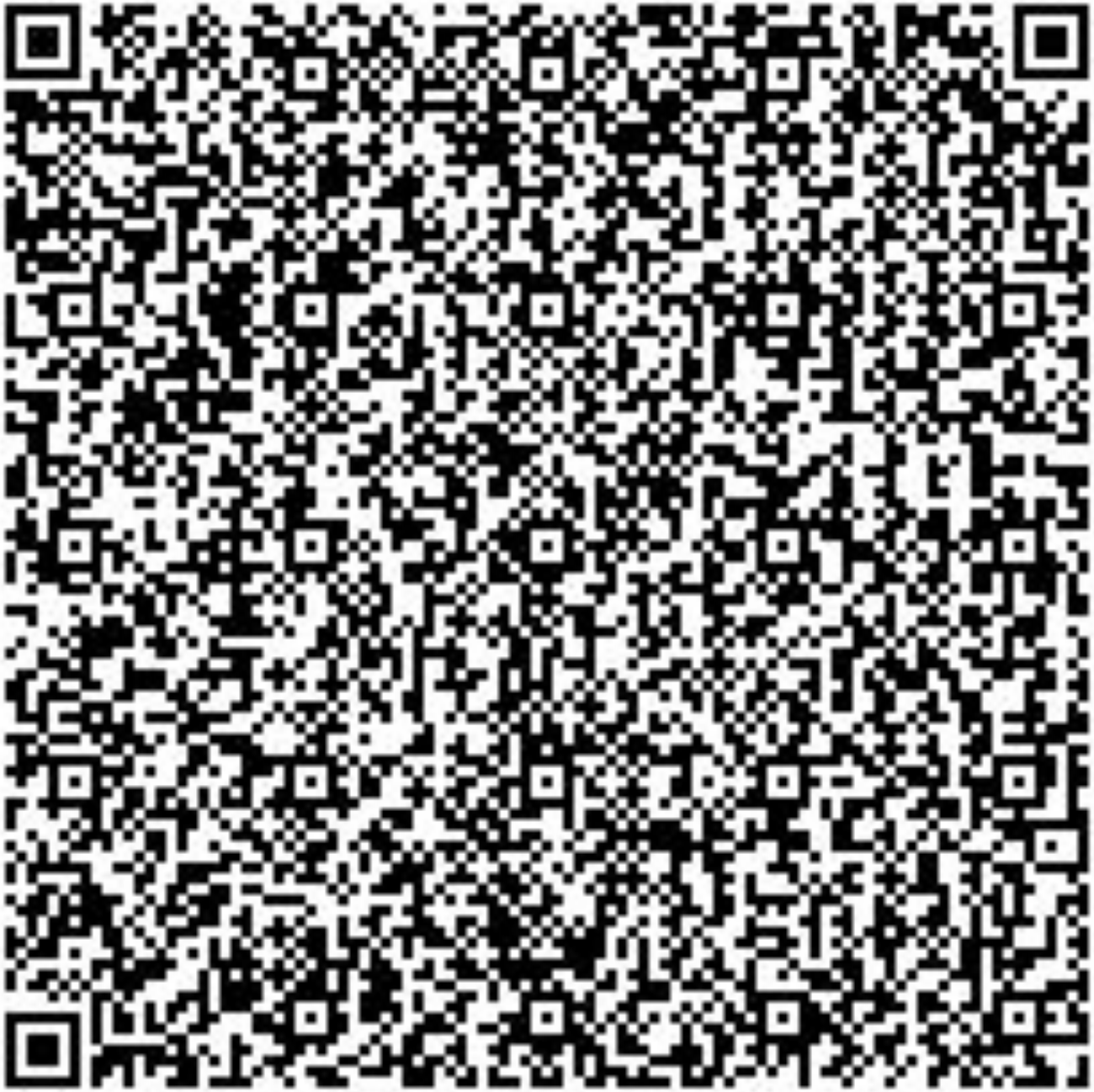
```
String name = "Шустрый";                    // имя
int health = 625;                           // здоровье
int mana = 820;                             // авторитет
int age = 21;                               // возраст
long longitude = ...;          // координаты местоположения
long latitude = ...;           // долгота и широта

String name2 = "Тормоз";                    // имя_второго
int health2 = 400;                          // здоровье_второго
int mana2 = 747;                            // авторитет_второго
int age2 = 24;                              // возраст_второго
long longitude = ...;          // координаты местоположения
long latitude = ...;           // долгота и широта
```

# What if there are two of them?

```
// тут алгоритм для каждого разный
public static void sayPhrase2() {
  System.out.println("Семки есть?");
}

// тут алгоритм одинаковый
public static long runToWest(int age, int health, long longitude)
{
    speed = health * age;
    return longitude - speed;
}
```

# What if there are two of them?

```
public class Main {

        public static void main(String args[]) {

                String name = "Шустрый";                        // имя
                int health = 625;                               // здоровье
                int mana = 820;                                         // авторитет
                int age = 21;                                   // возраст
                long longitude = ...;           // координаты местоположения
                long latitude = ...;                    // долгота и широта

                String name2 = "Тормоз";                        // имя_второго
                int health2 = 400;                              // здоровье_второго
                int mana2 = 747;                                // авторитет_второго
                int age2 = 24;                                          // возраст_второго
                long longitude2 = ...;          // координаты местоположения
                long latitude2 = ...;                   // долгота и широта

                sayPhrase1();
                if( runToWest(age, health) == runToWest(age2, health2)) { // встретились
                    sayPhrase2();
                }
        }

        public static void sayPhrase1() {
          System.out.println("Самые быстрые ноги на районе");
        }
        public static void sayPhrase2() {
          System.out.println("Семки есть?");
        }
        public static long runToWest(int age, int heath) {
                speed = health * age;
                return longitude - speed;
        }
}
```



# What if there are two of them?

```
public class Main {

        public static void main(String args[]) {

                String name = "Шустрый";                        // имя
                int health = 625;                               // здоровье
                int mana = 820;                                         // авторитет
                int age = 21;                                   // возраст
                long longitude = ...;           // координаты местоположения
                long latitude = ...;                    // долгота и широта

                String name2 = "Тормоз";                        // имя второго
                int health2 = 400;                              // здоровье_второго
                int mana2 = 747;                                // авторитет_второго
                int age2 = 24;                                          // возраст_второго
                long longitude2 = ...;          // координаты местоположения
                long latitude2 = ...;                   // долгота и широта

                sayPhrase1();
                if( runToWest(age, health) == runToWest(age2, health2)) { // встретились
                    sayPhrase2();
                }
        }

        public static void sayPhrase1() {
          System.out.println("Самые быстрые ноги на районе");
        }
        public static void sayPhrase2() {
          System.out.println("Семки есть?");
        }
        public static long runToWest(int age, int heath) {
                speed = health * age;
                return longitude - speed;
        }
}
```

**А что если их много?**

# First approach

```
String name[] = {  ...  };                    // имена
int health[] = ...;                           // здоровье
int mana[] = ...;                             // авторитеты
int age[] = ...;                              // возраста
long longitude[] = ...;          // координаты местоположения
long latitude[] = ...;           // долгота и широта
```

```
public static void sayPhrase1() {   }
public static void sayPhrase2() {   }
// ...
public static void sayPhraseN() {   }
```

**А что, если у каждого несколько имен?**

# Second approach

```
// собственный тип данных

class Guy {

   String name;                    // имя
   int health;                     // здоровье
   int mana;                       // авторитет
   int age;                        // возраст
   long longitude;                 // координаты местоположения
   long latitude;                  // долгота и широта
}

// Терминология:
// класс                           // Файл для класса
// поле                            // Значения по умолчанию
```

# Class contains properties and actions

```
// собственный тип данных
class Guy {

  // объявление остальных полей опущено
  String message = "Самые быстрые ноги на районе";

  public void sayPhrase() {            // методы без static
    System.out.println(message);
  }

  public long runToWest() {
      speed = health * age;
      return longitude - speed;
  }
}
```

# How to use it?

```
1.  public static void main(String args[]) {
2.      // экземпляр класса
3.      Guy guy = new Guy();
4.      guy.health = 1;   // обращение к атрибутам (св-вам)
5.      int guyMana = guy.mana;
6.      guy.sayPhrase ();
7.      if ( guy.runToWest() > 0 ) {
8.          System.out.println(“Hey, guys”);
9.      }
10. }
```

# How to use it?

```
public static void main(String args[]) {
    Guy faster = new Guy();
    Guy slower = new Guy();
    slower.sayPhrase ();
    if ( faster.runToWest() == slower.runToWest() ) {
        System.out.println("Hey, guy");
    }

    Guy [] guys = new Guy[50];
}
```

# Creation of class instances

```
Guy guy = new Guy();
```

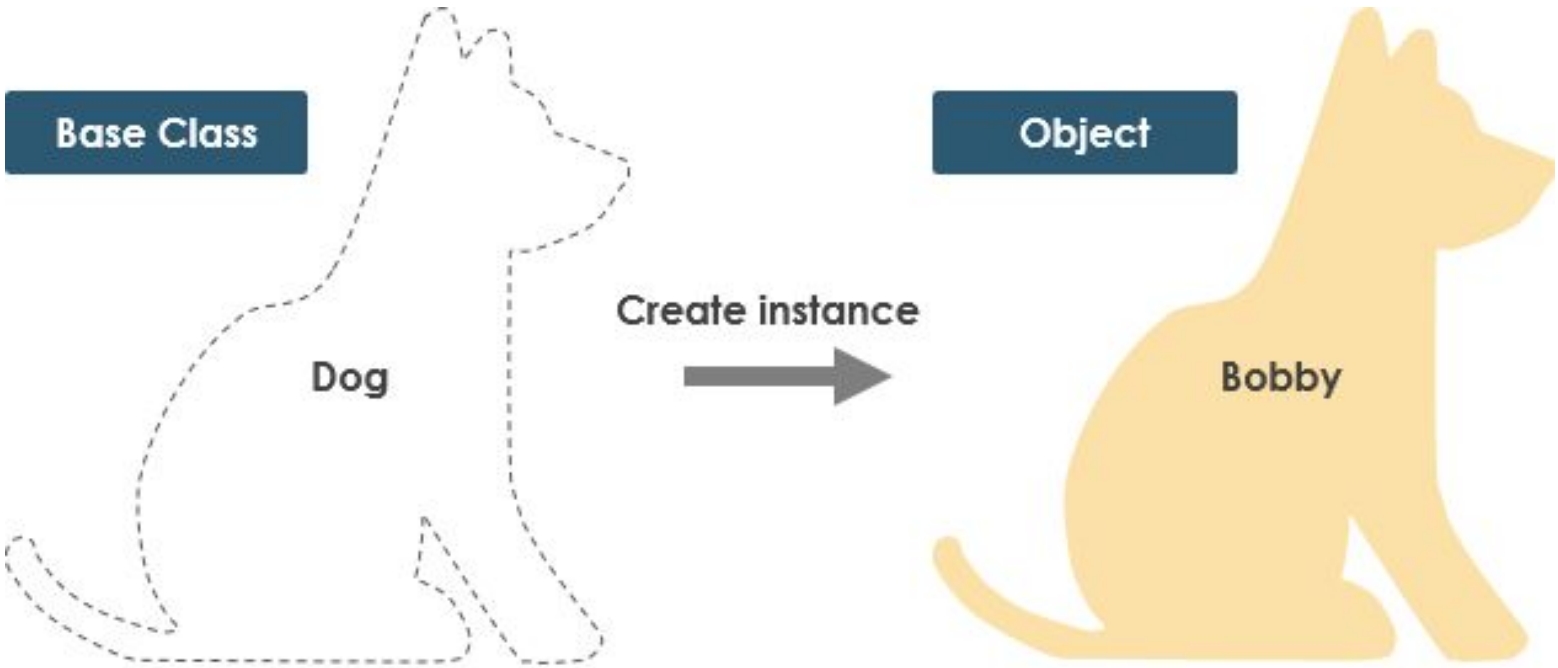
# Classes & Objects

```
class Guy {
   String name;
   int health;
   int mana;
   int age;
}
```

```
class Main {
    public static void main(String args[]) {

        Guy guy = new Guy();

    }
}
```

Ссылка на объект типа Guy, где Guy - это класс

# Dog bobby = new Dog();




| Properties | Methods |
|---|---|
| Color | Sit |
| Eye Color | Lay Down |
| Height | Shake |
| Length | Come |
| Weight | |

| Property Values | Methods |
|---|---|
| Color: Yellow | Sit |
| Eye Color: Brown | Lay Down |
| Height: 17 in | Shake |
| Length: 35 in | Come |
| Weight: 24 pounds | |

# Constructors

```
Guy guy = new Guy();
```

```
new Guy(); // new выделяет под объект память и… Guy()
```

**Что это такое и для чего?**

# Constructors

```
class Guy {

  String name;                    // имя - null
  int health;                     // здоровье - 0
  int mana;                       // авторитет - 0
  int age;                        // возраст - 0
  ...

  public Guy() {
    name = “Шустрый”;
  }

  public Guy(String newName) {
    name = newName;
  }
}
```

```
Guy guy = new Guy();

guy.name; // “Шустрый”

Guy guy = new Guy(“Жидкий”);

guy.name; // Жидкий
```

# Constructors

```
class Guy {

   String name;                          // имя - null
   int health;                           // здоровье - 0
   int mana;                             // авторитет - 0
   int age;                              // возраст - 0
   ...

   public Guy(String newName) {
      name = newName;
   }
}
```

```
Guy guy = new Guy(); // Error
```

# Constructors

```
class Guy {

   String name;                    // имя - null
   int health;                     // здоровье - 0
   int mana;                       // авторитет - 0
   int age;                        // возраст - 0
   ...

   // нет конструкторов
}
```

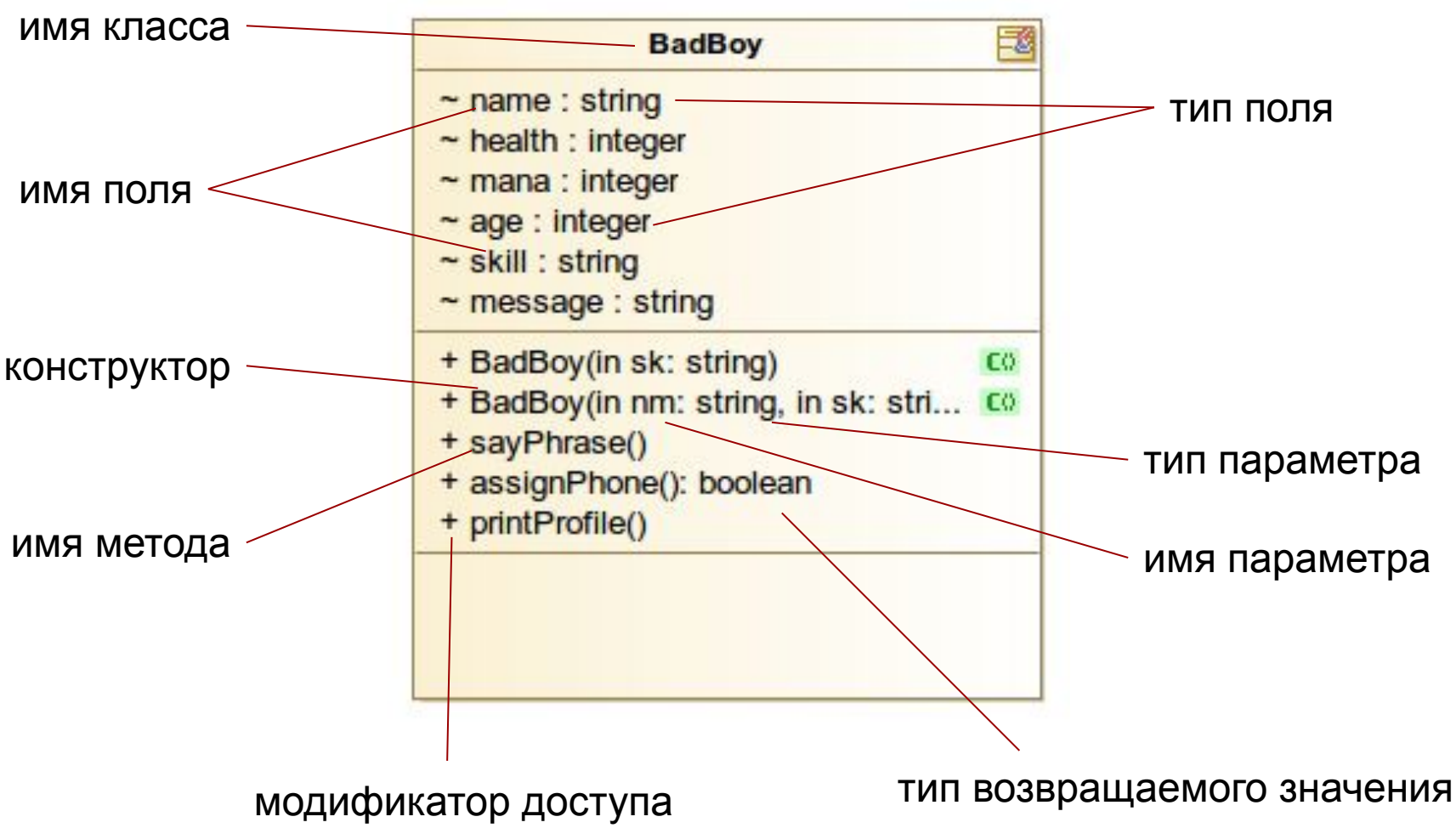
```
Guy guy = new Guy(); // OK

Guy guy = new Guy("Boy"); // Error
```

# Class description in UML

**Немного википедии**

**UML** (англ. Unified Modeling Language — унифицированный язык моделирования) — язык графического описания для объектного моделирования в области разработки программного обеспечения.

# Class description in UML



# Using your custom classes

```
import   java.util.Random;
import   Guy ?

public  class  Main {
  public  static  void  main (String []  args) {

    Random random = new Random();
    Guy guy = new Guy();

  }
}
```

**01) Для чего вообще писать import?**

**10) Откуда берутся названия java.lang, java.util и другие?**

# Packages

```
package ru.ifmo.se.prog.examples;

public class Guy {

  // все поля и методы класса

}
```

**Какие есть правила именования пакетов?**
**Влияет ли на класс (файл) наличие пакета?**

# Packages

```
import ru.ifmo.se.prog.examples.Guy;

public class Main {

    public static void main ( String[] args ) {

        Guy boy = new Guy();

        // ...
    }
}
```

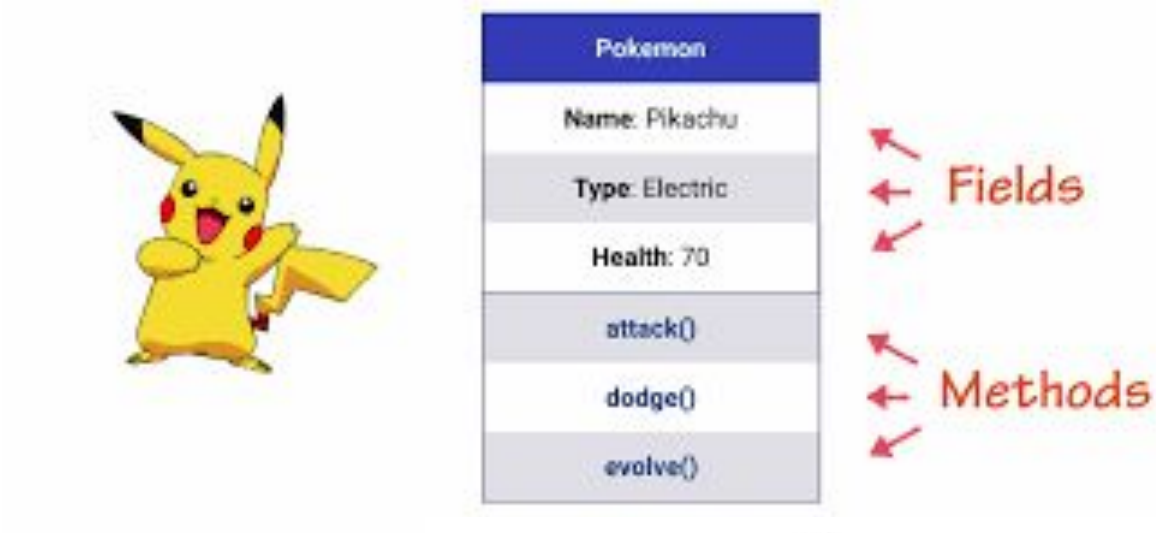
**А класс Main… он в каком пакете?**

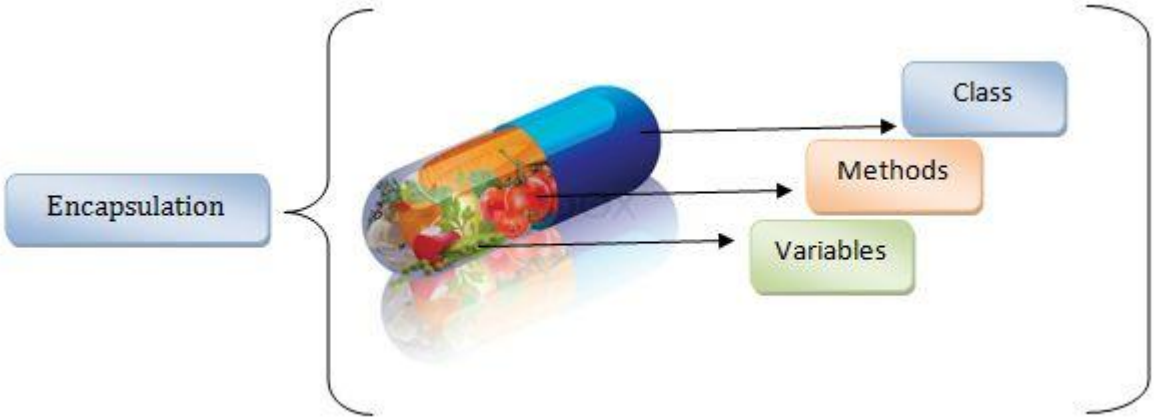
# Packages

```
package ru.ifmo.se.prog;

import ru.ifmo.se.prog.examples.BadBoy;

public class Main {

    public static void main ( String[] args ) {

        BadBoy veryBadBoy = new BadBoy();

        // ...
    }
}
```

Если классы находятся в одном пакете, то import не нужен.

# OOP principles



# Inheritance

Какими общими свойствами обладают два объекта?

# Inheritance

## Поиск одинаковых атрибутов

- имя
- возраст
- рост
- вес
- пол

## Поиск одинакового поведения

- сказать фразу
- применить навык
- присесть

# Inheritance

```
package ru.ifmo.se.prog.examples;

  public class Human {

    String name;                          // имя
    boolean isMale;
    int age;                              // возраст
    // … остальные свойства

    public void sayPhrase() { }
    // … остальные методы

  }
```

# Inheritance

```
package ru.ifmo.se.prog.examples;

public class Boy extends Human {

    public Boy() {
        isMale = true;
    }

    public void sayPhrase() {
        // ...
    }
}
```

# Inheritance

```
package ru.ifmo.se.prog.examples;

  public class Girl extends Human {

    public Girl() {
        isMale = false;
    }

    public void sayPhrase() {
        // ...
    }

  }
```

# Inheritance

```
package ru.ifmo.se.prog;

import ru.ifmo.se.prog.examples.*;

  public class Main {

    public static void main(String[] args) {
        Human[] array = new Human[2];
        array[0] = new Boy ();
        array[1] = new Girl ();

        for(Human human : array) {
            human.sayPhrase();
        }
    }
  }
```

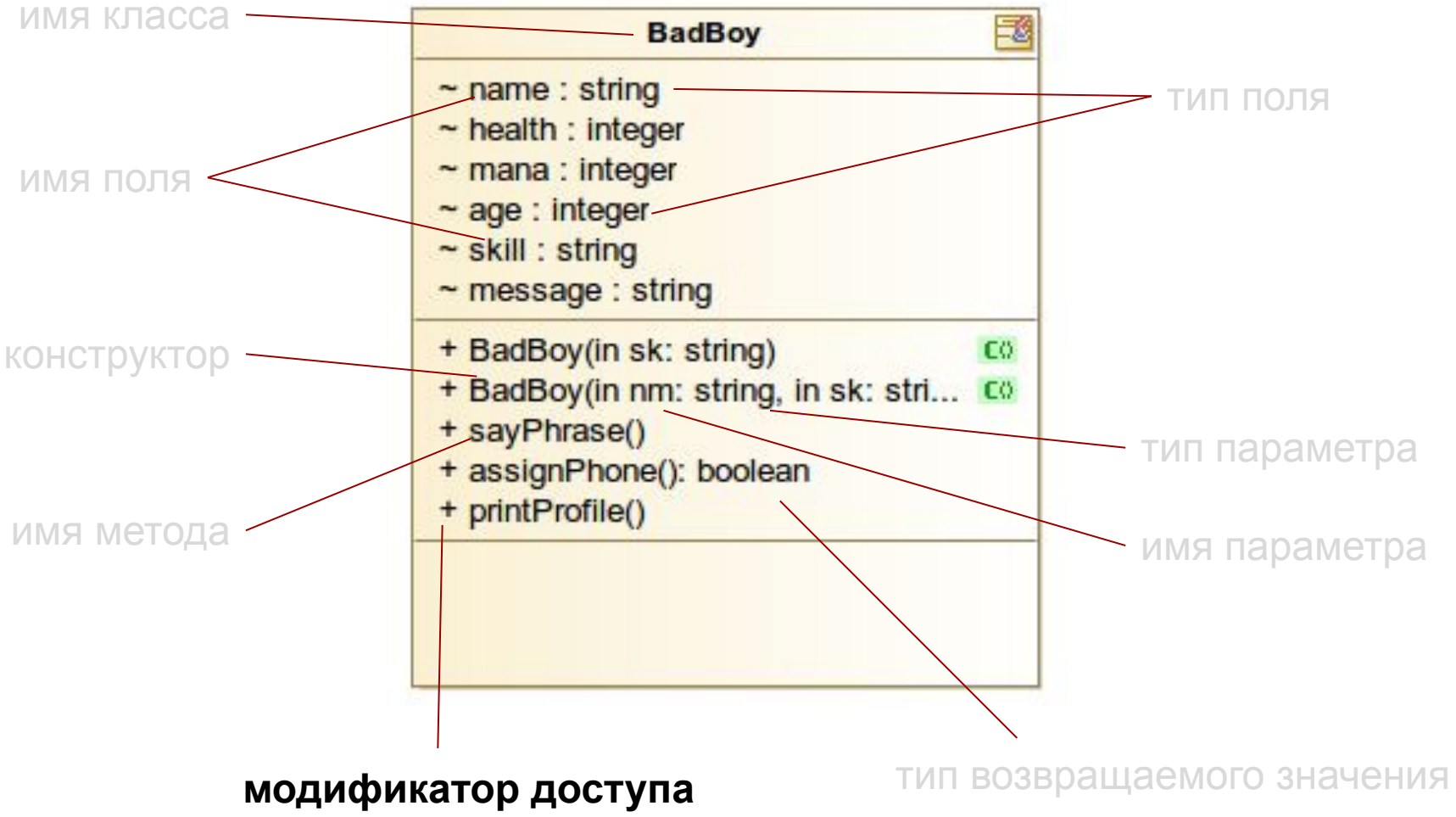
# Reference type VS Object type

```
package ru.ifmo.se.prog;

import ru.ifmo.se.prog.examples.*;

  public class Main {

    public static void main(String[] args) {
        Human human = new Boy();
    }
}
```

тип ссылки

тип объекта

# Class description in UML



# Access modifiers

public

protected

~~default~~

private

Применимо для

классы

методы

поля

конструкторы

# How to name the file?

```
package ru.ifmo.se.prog.examples;

class Boy {

  private int age;
  int mana;
  protected int health;
  public int height;

  // аналогично к методам и конструкторам. отсюда вопрос:

  private Boy() {}
}
```

WAT?? Пока можно просто подумать...

# How to name the file?

```
package ru.ifmo.se.prog.examples;

class Boy {

    // все поля и методы класса

}

class Girl {

    // все поля и методы класса

}
```

# How to name the file?

```
package ru.ifmo.se.prog.examples;

public class Boy {

  // все поля и методы класса

}

class Girl {

  // все поля и методы класса

}
```

Boy.java

# Questions?

**Faculty of Software Engineering and Computer Systems**

# Programming

## Lecture #3.
## OOP. Continue.

Instructor of faculty
Pismak Alexey Evgenievich
Kronverksky Pr. 49, 1331 room
pismak@itmo.ru

Saint-Petersburg

# В предыдущей серии

- Собственные типы данных - классы
- Наследование
- Реализация классов
- Создание экземпляров класса
- Конструкторы
- Модификаторы доступа

# В этой серии

- static
- this
- super
- Блоки инициализации
- Полиморфизм
- Инкапсуляция
- Интерфейсы
- Класс Object
- Интерфейсы/перечисления
- Абстрактные классы

# ‘static’

```
public class Guy {

  public static Guy createClone(Guy guy) {
      Guy clone = new Guy();
      clone.name = guy.name;
      clone.age = guy.age;
      return clone;
  }

  // где-то в main методе
  Guy myGuy = new Guy();
  Guy cloneOfMyGuy = Guy.createClone(myGuy);

  // new Guy().createClone()
}
```

# Why we wrote *'static'*

```
public class Main {

  public ~~static~~ void main(String args[]) {

  }
}
```

**JVM:**

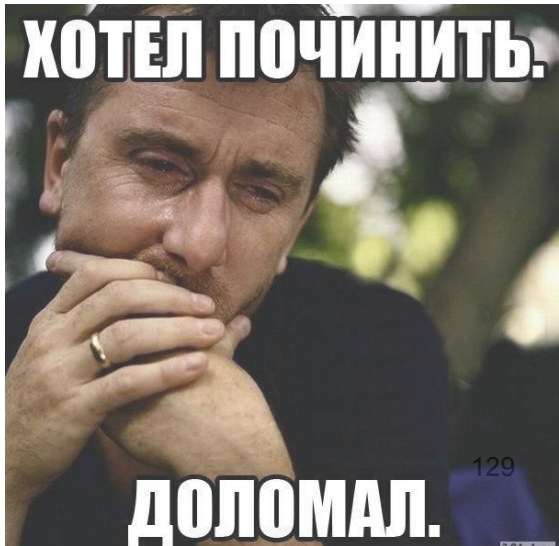
```
Main app = new Main();
app.main(args)
```

# Why we wrote *‘static’*

```
public class Main {

  public Main(int something) { ... }

  public static void main(String args[]) {

  }
}
```

**JVM:**

```
Main app = new Main(); // Error
```



# Why we wrote *'static'*

```
public class Main {

  public Main(int something) { ... }

  public static void main(String args[]) {

  }
}
```

**JVM:**
**Main.main(args)**

# static fields / static methods

Math.*PI* / Math.*E* / Math.*sin*() / Math.*log*()

System.*out* / System.*in* / System.*currentTimeMillis*()

Byte.*MIN_VALUE* / Byte.*valueOf*() / Byte.*parseValue*()

```
class Item {
  private static int itemID = 0;
  public Item() { itemID++; }
}
```

# ‘this’ and ‘super’

```
package ru.ifmo.se.prog.examples;

  public class Human {

    String name;                          // имя
    boolean isMale;                       // пол. true если муж.
    int age;                              // возраст
    // … остальные свойства

    public Human() {
        // сначала бы вызвать родительский конструктор...
    }
    public void applySkill(int age) { }
    // … остальные методы

  }
```

# ‘this’ and ‘super’

```
package ru.ifmo.se.prog.examples;

  public class Human {

    String name;                          // имя
    boolean isMale;                       // пол. true если муж.
    int age;                              // возраст
    // … остальные свойства

    public Human() {
        // сначала бы вызвать родительский конструктор...
    }
    public void applySkill(int age) { this.age = age; }

  }
```

# ‘this’ and ‘super’

```
package ru.ifmo.se.prog.examples;

  public class Human {

    String name;                          // имя
    boolean isMale;                       // пол. true если муж.
    int age;                              // возраст
    // … остальные свойства

    public Human() {}
    public Human(int age) {
        this();
    }
    public void applySkill(int age) { this.age = age; }

  }
```

# ‘this’ is a first implicit method parameter

```
package ru.ifmo.se.prog.examples;

  public class Human {
    String name;

    public String getName(Human this) { return this.name; }
    public void setName(Human this, String name) {
        this.name = name;
    }
}
```

# ‘this’ and ‘super’

```
package ru.ifmo.se.prog.examples;

  public class BadBoy extends Human {

    public BadBoy() {
        super(“Name”);
    }

  }
```

# Init blocks

```
1. public class A {
2.  public A() {
3.   System.out.println("A constr");
4.  }
5.
6.  {
7.   System.out.println("A block");
8.  }
9. }
```

```
1. public class B extends A{
2.  public B() {
3.   System.out.println("B constr");
4.  }
5.
6.  {
7.   System.out.println("B block");
8.  }
9. }
```

```
public
  static
    void main(String ...s) {

        B b = new B();
}
```

A block
A constr
B block
B constr

# Init blocks

```
 1. public class Main {
 2.
 3.   private static int field;
 4.
 5.   public static void main (String... s) {
 6.       System.out.println("main");
 7.   }
 8.
 9.   static {
10.       field = 10;
11.       main(null);
12.       System.exit(-1);
13.   }
14.
15. }
```

# Polymorphism

Кто умеет пользоваться этой штукой?

# Polymorphism

Кто умеет пользоваться этой штукой?

# Polymorphism

Кто умеет пользоваться этой штукой?

# Polymorphism

Кто умеет пользоваться этой штукой?

# Polymorphism

Кто умеет пользоваться этой штукой?

# Polymorphism

это возможность использовать объекты с одинаковым протоколом взаимодействия без информации о типе и внутренней структуре объекта

1. Переопределение методов
2. Перегрузка методов
3. Ковариантны возвращаемых типов
4. Абстрактные классы

# Method overriding

```
public class Vector {

    protected float x, y;

    public void multiply(float value) { }
    public void move(Vector v) { }
    public Vector normalize() { ... }
}


public class Vector3 extends Vector {
    protected float z;

    @Override
    public void multiply(float value) { }
}
```

# Method overloading

```
public class Vector {
  …
  public void move(Vector v) { }

}



public class Vector3 extends Vector {

  protected float z;

  public void move(Vector3 vector3){ //это другой метод move
     …
  }
}
```

# Covariance

```
public class Vector {
    …
    public Vector normalize() { ... }
}



public class Vector3 extends Vector {

    protected float z;

    @Override
    public Vector3 normalize() { …  }
}
```

# Abstract classes

```
public abstract class Animal {

    private String name;
    private int health;

    public Animal() { ... }

    public void eat(Food food) {
      health += food.getCalories();
    }

    public abstract void run();
    public abstract void jump();

}
```

~~Animal animal = new Animal();~~

Даже если этих методов нет

# Abstract classes

```java
public class Crocodile extends Animal {

    @Override
    public void run() {
        // do nothing
    }

    @Override
    public void jump() {
        // do nothing too
    }
}
```

# Abstract classes

```
public class Kangaroo extends Animal {

    @Override
    public void run() {
        while(true) jump();
    }

    @Override
    public void jump() {
        // code of jump
    }
}
```

# Abstract classes

```
public abstract class Felidae extends Animal {

    public abstract void hunt();

}
```

# Abstract classes

```
public static void main(String[] args) {

        Animal[] animals = new Animal[5];

        // initialization array here
       // animals[0] = new Crocodile();

        for(Animal animal : animals) {
            animal.run();
        }
    }
```

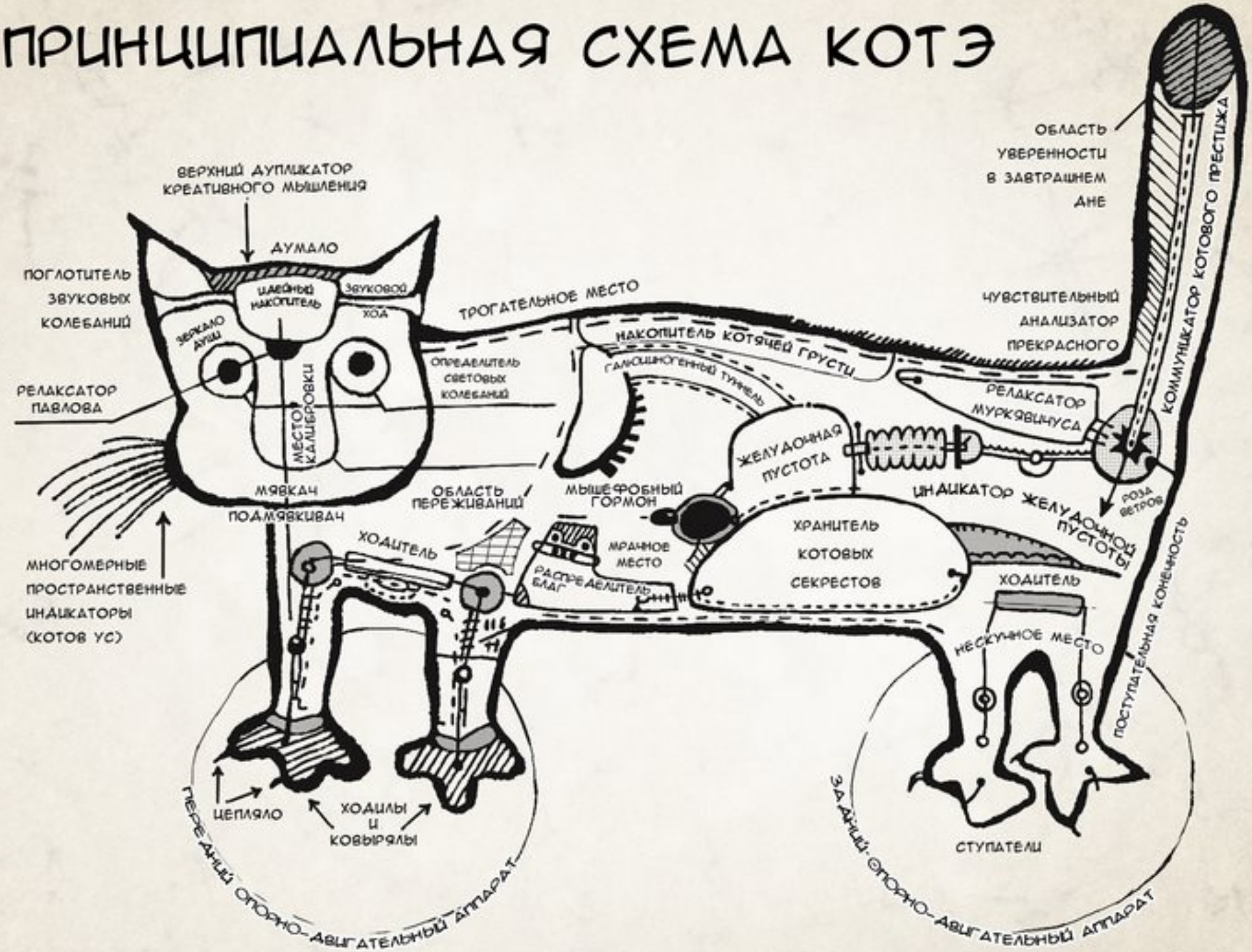
ПОЛИМОРФИЗМ

# Final modifier

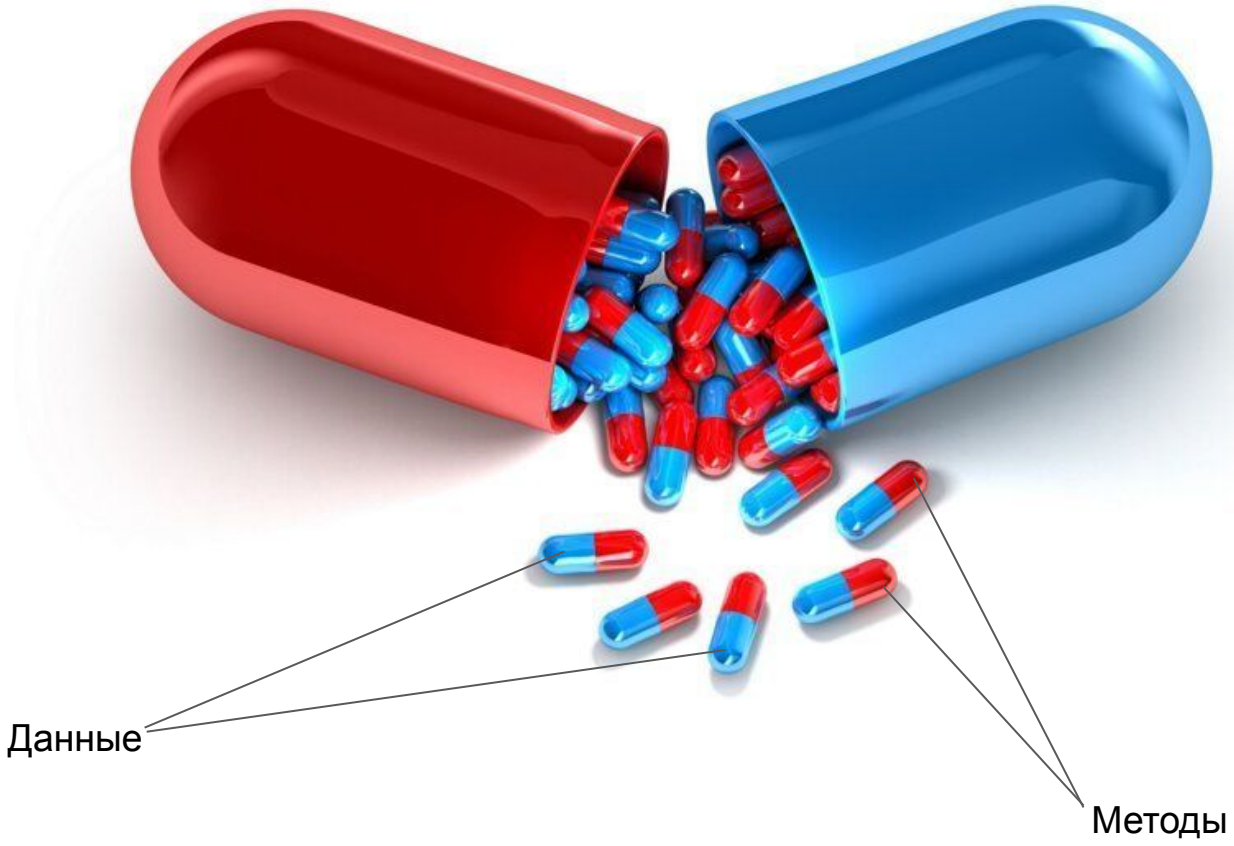
```
final class  // cannot be extended

final method() // cannot be overridden

final variable   // cannot be reassigned
```

# Encapsulation

# ПРИНЦИПИАЛЬНАЯ СХЕМА КОТЭ



# Encapsulation




Это свойство объекта, объединяющее данные и методы, работающие с ними, в классе и скрывающее детали реализации.

Инкапсуляция связана с понятием интерфейса класса. Всё, что не входит в интерфейс, инкапсулируется в классе.

# Interfaces

1. Поведение объектов
2. Реализация интерфейса
3. Зачем интерфейсы, если есть классы?

# Interfaces

```
public interface Felidae {                    // кошачьи

    public static final String DEFAULT_NAME = "Барсик";

    void feed();
    Reaction stroking();

}
```

1. Что это вообще такое?
2. Все методы абстрактные
3. Все методы public
4. Все “поля” интерфейса могут быть только public static

# Felines (Кошачьи)

```
public class Cat implements Felidae {

    private String name = Felidae.DEFAULT_NAME ;
    private Size size = new Size(10, 20, 40, 22);

    @Override
    public void feed() {
        this.size.increase();
    }

    @Override
    public Reaction stroking() {
        return new Reaction.POSITIVE;
    }

}
```

# Felines (Кошачьи)

```
public class Lion implements Felidae {

    private String name = "Симба";
    private Size size = new Size(100, 200, 400, 220);

    @Override
    public void feed() {
        this.size.increase();
    }

    @Override
    public Reaction stroking() {
        return new Reaction.TO_KILL_AND_EAT;
    }

}
```

# Advantages of interfaces

1) Абстрагирование от реализации

```
class Human {

    public void feedTo ( Felidae felidae ) {
        felidae.feed();
    }

}
```

Чем это отличается от использования абстрактных классов?

# Advantages of interfaces

2) Множественное наследование

```
public class Cat implements Felidae, Toy {

    …

    @Override
    public void play() {
        this.size.decrease();
    }

}
```

# Interface methods modifiers

1. abstract
2. default
3. static
4. private

# Check objects type

```java
public interface Toy {
    void play();
}
```

```java
public class Cat implements Toy {

    …

    @Override
    public void play() {
        this.size.decrease();
    }

    public void play(Cat cat) {
        // ...
    }
}
```

```java
Toy toy = new Cat();
method(toy);


void method(Toy toy){
    toy.play();

    toy.play(new Cat());
}
```

# Check objects type

```
Toy toy = new Cat();
method(toy);
```

---

```
1.  void method(Toy toy){
2.      toy.play();
3.
4.      if (toy instanceof Cat)
5.          ((Cat)toy).play(new Cat());
6.  }
```

---

```
1.  void method(Toy toy){
2.      toy.play();
3.
4.      if (toy.getClass() == Cat.class)
5.          Cat cat = (Cat)toy;
6.          cat.play(new Cat());
7.  }
```

# Check objects type

```
Toy toy = new Cat();
method(toy);
```

---

```
1.  void method(Toy toy){
2.      toy.play();
3.
4.      if (toy instanceof Cat cat) {
5.          cat.play(cat);
6.      }
7.  }
```

# Object

1. `int hashCode`
2. `boolean equals`
3. `Object clone`
4. `void finalize`
5. `String toString`
6. `wait / notify / notifyAll`

ВОПРОСЫ НА ЛАБОРАТОРНЫХ

# toString

```
public class Cat {

    private String name = …;

    public String toString() {
        return "Cat_" + name;
    }
}
```

"Cat@382bf1c8"

---

```
public static void main (...) {
    System.out.println( new Cat() );
}
```

Перегруженный метод

# equals

cat == new Cat()

```
public class Cat {

   private String name = …;

   public boolean equals(Object o) {
      if (this == o) return true;
      if (o == null || getClass() != o.getClass()) return
false;
      Cat cat = (Cat) o;
      return Objects.equals(name, cat.name);
   }
}
```

---

```
public static void main (...) {
   Cat cat = new Cat();
   System.out.println( cat.equals( new Cat()) );
}
```

# hashCode()

```
public class Cat {

   private String name = …;
   private int age = …;

   public int hashCode() {
      return (name.hashCode() << 4) + age;
   }
}
```

a.equals(b) == true

a.hashCode == b.hashCode()

---

```
public static void main (...) {
   Cat cat = new Cat();
   System.out.println( cat.hashCode() );
}
```

# empty class ?

```
public class MyClass                    {



}
```

# implicit code

```
import java.lang.*;

public class MyClass extends Object {

  public MyClass() {
    super();
  }
}
```

# Enum

```
public class SignalTraffic {

    private String color;
    private boolean state;

    public SignalTraffic(String color) {
        this.color = color;
    }

    public void changeState() {
        this.state = !state;
    }
}
```

# Enum

```
public class Main {

    public static void main(String … args) {
        SignalTraffic red = new SignalTraffic("RED");
        red.change();
    }
}
```

Как переписать `SignalTraffic` так, чтобы существовало только три объекта с фиксированно заданными полями?

# Enum

```
public enum SignalTraffic {

    RED,
    YELLOW,
    GREEN

}
```

---

```
public class Main {

    public static void main(String … args) {
        SignalTraffic red = SignalTraffic.RED;

    }
}
```

# Enum (upgrade#1)

```
public enum SignalTraffic {
    RED("красный"),
    YELLOW("желтый"),
    GREEN("зеленый");

    private String name;
    SignalTraffic(String name) { this.name = name; }
    public String getName();
}
```

поля
конструкторы
методы

---

```
public static void main(String … args) {
   SignalTraffic red = SignalTraffic.RED;
   System.out.println("Color of signal = " + red.getName());
}
```

# Enum (upgrade#2)

```
public enum SignalTraffic {
    RED("красный"),
    YELLOW("желтый"),
    GREEN("зеленый") {
      public void blink() {}
    };

    private String name;
    SignalTraffic(String name) { this.name = name; }
    public String getName();

    public void glow() { }
}
```

# Enum (upgrade#3)

```
public enum SignalTraffic {
    RED("красный"),
    YELLOW("желтый"){
        @Override
        public void glow() { }
    },
    GREEN("зеленый");

    ...

    public void glow() { }
}
```

# Enum (upgrade#4)

```
public enum SignalTraffic implements Glowable {
    RED("красный"),
    YELLOW("желтый"){
        @Override
        public void glow() { }
    },
    GREEN("зеленый");

    ...

    public void glow() { }
}
```

# Enum “under the hood”

```
public class SignalTraffic extends Enum {
    SignalTraffic RED = new SignalTraffic(“красный”);
    SignalTraffic YELLOW = new SignalTraffic(“желтый”);
    SignalTraffic GREEN = new SignalTraffic(“зеленый”);

    ...

}
```

Как следствие:

- от enum нельзя наследоваться
- Enum уже притаскивает некоторые методы по наследству:
  - name
  - ordinal
  - valueOf
  - values